

Architecture Evolution of Google Guava

EPL484 Software Evolution – Project 1 – Team X

⟨Member 1 (ID)⟩, ⟨Member 2 (ID)⟩

October 2026

- [1. Project choice and characteristics](#)
 - [1.1 What is Guava?](#)
 - [1.2 Eligibility \(Phase 1\)](#)
 - [1.3 Number and size of versions](#)
 - [1.4 Is the architecture documented?](#)
- [2. Methodology](#)
 - [2.1 Data collection](#)
 - [2.2 Data cleaning and version selection](#)
 - [2.3 Tools and libraries](#)
 - [2.4 Implementation steps, k-means features and choice of k](#)
 - [2.5 Quality metrics](#)
 - [2.6 Additional analyses from the lectures](#)
- [3. Results and discussion](#)
 - [3.1 Size](#)
 - [3.2 Class dependencies](#)
 - [3.3 Widely-used \(noise\) classes](#)
 - [3.4 Recovered architectures \(A1-A4\)](#)
 - [3.5 Quality of the architecture over time](#)
 - [3.6 Distance to the reference architecture \(MoJoFM\)](#)
 - [3.7 Omnipresent classes: JNode vs Bunch](#)
 - [3.8 Architectural smells and their evolution](#)
 - [3.9 Lehman's laws: dependency changes and work rate](#)
- [4. Architecture recovery with AI \(latest version only: 33.7.0\)](#)
- [5. Reflection](#)
 - [5.1 Comparing the architectures of the latest version](#)
 - [5.2 Does the AI tool help? How do information and prompts influence the result?](#)
 - [5.3 Which method for other systems?](#)
- [6. Threats to validity](#)
- [References](#)
- [Appendix A – Full metric tables](#)

Code and data: GitHub repository `ep1484.fall26.project1.teamX`, folder `guava/` (data in `guava/data/`, results in `guava/results/`, AI material in `guava/ai/`); the analysis scripts in `scripts/` are shared and run with `PROJECT=guava`. Every table in this report is generated from the result files by `scripts/report_tables.py`.

1. Project choice and characteristics

1.1 What is Guava?

[Guava](#) is Google's set of core libraries for Java. It is used in most Java projects inside Google and in a large part of the Java ecosystem. It offers:

- **Collections** – immutable collections (`ImmutableList`, `ImmutableMap`, ...), new collection types (`Multimap`, `Multiset`, `BiMap`, `Table`), ranges (`Range`, `RangeSet`), and utilities (`Lists`, `Maps`, `Sets`, `Iterables`, `FluentIterable`, `Ordering`);
- **Base utilities** – `Preconditions`, `Optional`, functional types, string handling (`Joiner`, `Splitter`, `CharMatcher`), `Stopwatch`;
- **Concurrency** – `ListenableFuture` and the Futures combinators, executors, `Service`, `RateLimiter`;
- **Caching** (`CacheBuilder`, `LoadingCache`), a **graph library** (`Graph`, `ValueGraph`, `Network`), **I/O** (`ByteSource`, `CharSource`, `Files`), **hashing** (`Hashing`, `BloomFilter`), **primitives and math**, **reflection** (`TypeToken`, `ClassPath`), an **event bus**, **escaping** and **networking** helpers (`MediaType`, `InternetDomainName`).

Guava is a **library**: it has no runtime architecture of its own, so its architecture is its static decomposition into packages and classes and the dependencies between them.

1.2 Eligibility (Phase 1)

Criterion	Guava
≥ 10 major/minor versions	54 major/minor releases (33 major v1.0–v33.0, 21 minor), plus 28 patch releases and 25 release candidates (109 v* tags)
Repository on GitHub	google/guava
Latest version ≥ 10 K LOC	33.7.0: 611 production .java files, 96,646 non-comment, non-blank LOC
Not archived / not a fork	✓ / ✓
≥ 50 commits since 1/1/2024	1,261 (of 7,546)
Created before 2024	history starts 2009-06-18

1.3 Number and size of versions

We analyse a **sparse sample of 13 releases**: every second major release from 10.0 to 32.0 plus the latest feature release 33.7.0. This covers 15 years (September 2011 – August 2026), and no two sampled releases are consecutive. Table 1 gives the size of each version. *Class files* include inner and anonymous classes. *Top-level classes* are compilation units. *Dependencies* are distinct dependencies between top-level classes.

Table 1 – Size of the analysed versions.

Version	Date	Packages	Class files	Top-level classes	Δ classes	Dependencies	Deps/class
10.0	2011-09-27	10	1157	317	-	1289	4.16
12.0	2012-04-30	13	1332	373	+17.7%	1597	4.39
14.0	2013-02-25	13	1584	412	+10.5%	1945	4.87
16.0	2014-01-17	17	1663	448	+8.7%	2121	4.88
18.0	2014-08-25	17	1677	454	+1.3%	2112	4.81
20.0	2016-10-28	18	1799	514	+13.2%	2453	4.92
22.0	2017-05-22	18	1850	529	+2.9%	2534	4.93
24.0	2018-02-01	18	1922	549	+3.8%	2633	4.93
26.0	2018-08-01	18	1936	556	+1.3%	2686	4.96
28.0	2019-06-11	18	1932	554	-0.4%	2699	5.01
30.0	2020-10-16	18	2010	569	+2.7%	2769	5.01
32.0.0	2023-05-26	18	1997	607	+6.7%	2821	5.06
33.7.0	2026-08-17	18	1948	594	-2.1%	2750	4.96

1.4 Is the architecture documented?

Only at the API level. The README and the [user guide](#) describe the packages feature by feature (“Collections”, “Caches”, “Functional idioms”, “Concurrency”, “Strings”, “Primitives”, “Ranges”, “I/O”, “Hashing”, “EventBus”, “Math”, “Reflection”, “Graphs”), and the Javadoc documents each package. The wiki also documents policies with architectural consequences: a strict **API-compatibility** policy, where @Beta APIs may change and the rest are only removed after deprecation, and the separate **JRE and Android flavours**. There is **no component/dependency view** and no description of the internal structure of the large packages. The package structure *is* the documented architecture.

2. Methodology

The pipeline is the same as for our first system (Spring AI), shared through the scripts in scripts/. Only the data collection is Guava-specific (guava/scripts/).

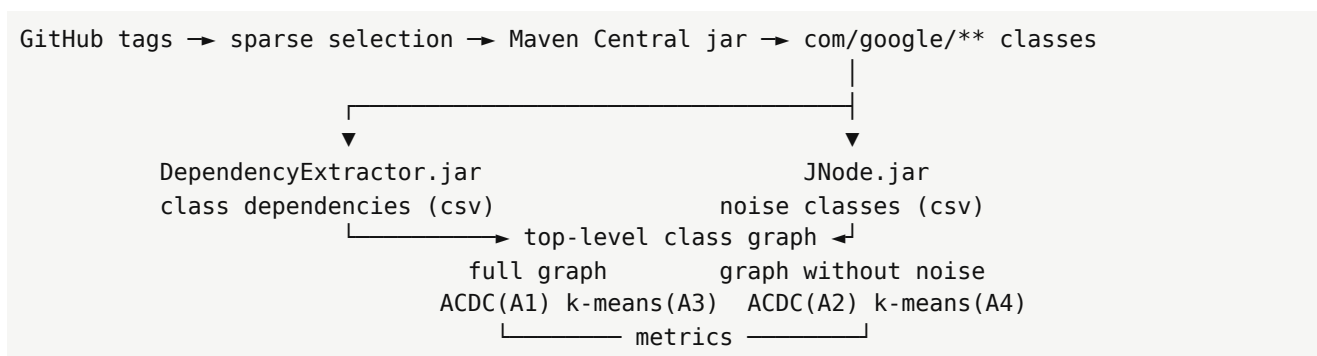


Figure 1 – Analysis pipeline.

2.1 Data collection

1. **Releases.** All tags were read from a blob-less clone of the repository with their creation dates (guava/data/tags.csv). The Phase 1 statistics come from the same clone (commit counts, NCLOC of guava/src at v33.7.0).
2. **Binaries.** Guava is a single Maven artifact (com.google.guava:guava). guava/scripts/collect_versions.py downloads the jar of every selected release from Maven Central. From 23.1 on Guava is published in two flavours (-jre, -android); we use the main -jre flavour. The script keeps the com/google/** classes (Guava's own code, including com.google.thirdparty.publicsuffix) and places the jar in guava/data/workspace/<v>/bin/, the input layout of the course tools.

2.2 Data cleaning and version selection

- **Population.** The major/minor releases from 10.0 on. Before 10.0 Guava used the r01...r09 numbering, and 10.0 is its first release with the current scheme.
- **Excluded releases.** Patch releases (x.y.z, $z > 0$), release candidates (-rc) and two special tags (13.0-final, 15.0-cdi1.0).
- **Sparse sample.** Every second major release (10, 12, ..., 32) and the latest release 33.7.0: **13 versions.** The time gaps vary from 6 months to 3 years (Table 1) because Guava's release rate slowed down after 2020; the version axis of all figures is ordinal, not time-proportional.
- **Excluded classes.** module-info/package-info; inner/anonymous classes are folded into their top-level class. 5-10 classes per version have no dependency to or from another Guava class (e.g. Charsets, Flushables, Runnables, which only use JDK types) and are left out of A1-A4.

2.3 Tools and libraries

Step	Tool / library
Class dependencies (3.2)	Course DependencyExtractor.jar
Noise classes (3.3)	Course JNode.jar with the documented performance and precision patches from our first analysis (tools/JNode-fast-p4.jar, identical results where the original works; see tools/jnode-patch/)
ACDC (A1, A2)	ACDC (Tzerpos & Holt), Java port from USC ARCADE (tools/acdc.jar)
Clustering (A3, A4)	scikit-learn KMeans, TruncatedSVD; kneed (elbow)
Metrics, plots	Python 3.13, numpy, scipy, matplotlib

JNode threshold. JNode flags a class if its normalised SIG $\geq \text{mean} + 1\sigma$. For 10.0 and 12.0 this limit (1.010 and 1.023) lies above the maximum SIG (1.0), so no class is flagged. For these two versions we clamp the limit to the maximum (the classes with the highest SIG are noise).

ACDC naming. ACDC names a subsystem after the base name of its dominator (text before the last dot). For Java class names that is the package, which merges all subsystems of a package in the output. We therefore give ACDC file-style names (pkg.Class.java) and strip

the suffix from the output. Without this correction ACDC appeared to return only 9-14 “clusters” for Guava, which were simply its packages.

2.4 Implementation steps, k-means features and choice of k

recover.py builds the class graph of each version, writes the two RSF files (guava/data/rsf/), runs **A1/A2** (ACDC on the full graph and on the graph without noise classes) and **A3/A4** (k-means on the same graphs), and evaluates them together with the **package decomposition (PKG)**.

Features for k-means. Each class is represented by its dependency profile (row of $A + A^T + I$), L2-normalised and reduced to 64 dimensions with truncated SVD, so that k-means approximates clustering by cosine similarity of dependency profiles.

Choice of k. For $k = 4 \dots n/3$ we recorded inertia and silhouette, and chose k with the **elbow method** (Kneedle). The silhouette peaks at clusters of only 3-4 classes, which is too fine to be an architecture. The elbow gives clusters of 6-9 classes, and at the elbow the silhouette already reaches 92-97 % of its maximum (Figure 2). k grows from 52 (A3) / 42 (A4) in 10.0 to 69 / 64 in 33.7.0.

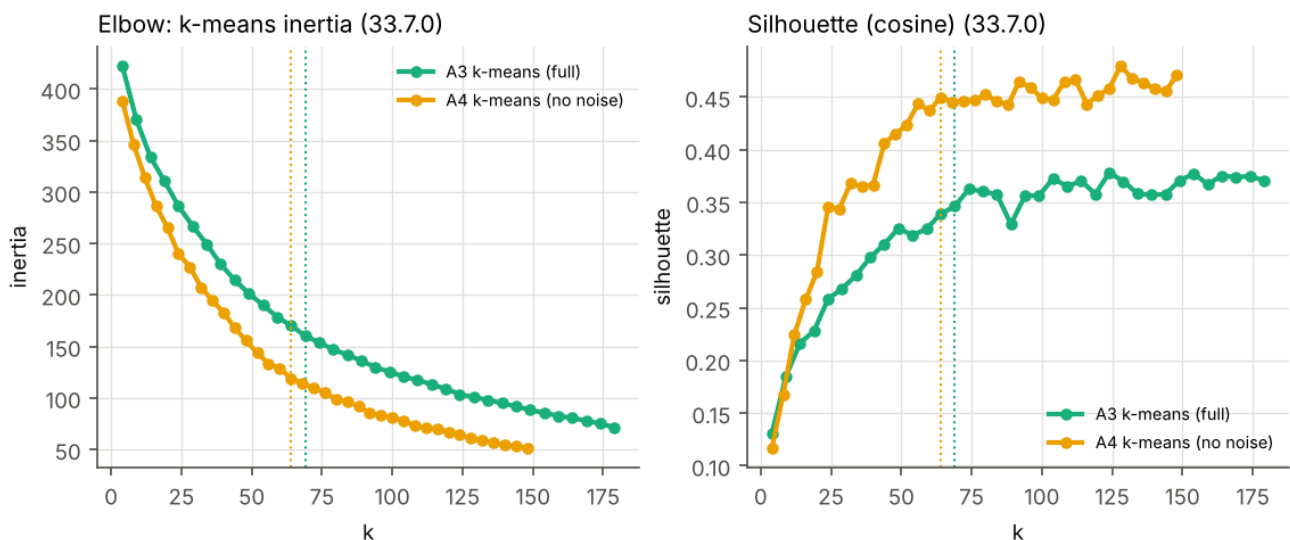


Figure 2 - k selection for 33.7.0: elbow (left) and silhouette (right); dotted lines = chosen k.

2.5 Quality metrics

For a clustering into k clusters, with μ_i the dependencies inside cluster i (N_i classes) and ε_{ij} the dependencies between clusters i and j (directed, unweighted class graph), we use the definitions of the course (Lecture 6-7, “Ποια θεωρείται καλή διάτμηση”):

- **Cohesion** (intra-connectivity) $A_i = \mu_i / N_i^2$. The lecture allows μ_i/N_i^2 or $\mu_i/(N_i(N_i-1))$. We use N_i^2 because it is defined for one-class clusters (packages and ACDC produce some). We report the mean over clusters.
- **Coupling** (inter-connectivity) $E_{ij} = \varepsilon_{ij} / (2 \cdot N_i \cdot N_j)$, averaged over all cluster pairs.
- **MQ - Modularization Quality** (course definition, Mancoridis et al. 1998): $MQ = (1/k) \cdot \sum A_i - (2/(k(k-1))) \cdot \sum E_{ij}$, in $[-1, 1]$. This is the **main quality measure** of the

assignment. Because A_i divides by N_i^2 , MQ favours many small, dense clusters, so it is only comparable between architectures with a similar number of clusters.

- Supplementary measures, independent of k : **TurboMQ/ k** , the mean cluster factor $2\mu_i/(2\mu_i+\varepsilon_i)$ of the later Bunch MQ (Mitchell & Mancoridis 2006), where ε_i counts all dependencies crossing the border of cluster i ; and **intra deps**, the share of all dependencies that stay inside a cluster. TurboMQ (the sum, which grows with k) is given in the appendix.

2.6 Additional analyses from the lectures

- **MoJoFM** (Lecture 6-7): $\text{MoJoFM}(A, R) = (1 - \text{mno}(A, R) / \max \text{mno}(\cdot, R)) \cdot 100 \%$, where mno is the minimum number of *Move* and *Join* operations that turn architecture A into reference R (100 % = identical). It is computed with the original MoJo 2.0 implementation (`tools/mojo.jar`). The system has no expert (“ground-truth”) architecture, so we use the developers’ **package structure** as the reference for every version and, for the latest version, also the **AI architecture** of Phase 4. MoJoFM between the architectures of consecutive versions measures **architectural stability**; we also report the adjusted Rand index (ARI).
- **Omnipresent classes with the Bunch rule** (Lecture 6-7): $\text{in-degree} > 3 \times \text{average in-degree}$. We compare it with JNode (Constantinou et al. 2015), and, as in the lecture’s comparison of “System”, “Noise” and “Bunch”, we measure how close ACDC gets to the reference architecture without each set.
- **Architectural smells and their evolution** (Lecture 5; Fontana et al. 2016, Sas et al. 2019), each normalised by the number of classes or packages: *cyclic dependency* (packages/classes in a dependency cycle, i.e. a strongly connected component of size > 1 , found by depth-first search); *hub-like dependency* (classes with fan-in and fan-out above the system medians and $|\text{fan-in} - \text{fan-out}| < 0.25 \cdot (\text{fan-in} + \text{fan-out})$); *unstable dependency* (packages, instability $I = C_e/(C_a+C_e)$, of which more than 30 % of the package dependencies point to less stable packages – the Arcan threshold); *god component* (packages with more classes than mean $+ 2\sigma$ of the package sizes, since the lecture leaves the threshold open).
- **Lehman’s laws** (Lecture 1-2): besides size and dependency growth, we count the **dependencies added and removed** per version (as in the lecture’s Eclipse example for law II) and the **commits per month** between the analysed releases (law IV, conservation of organisational stability).

3. Results and discussion

3.1 Size

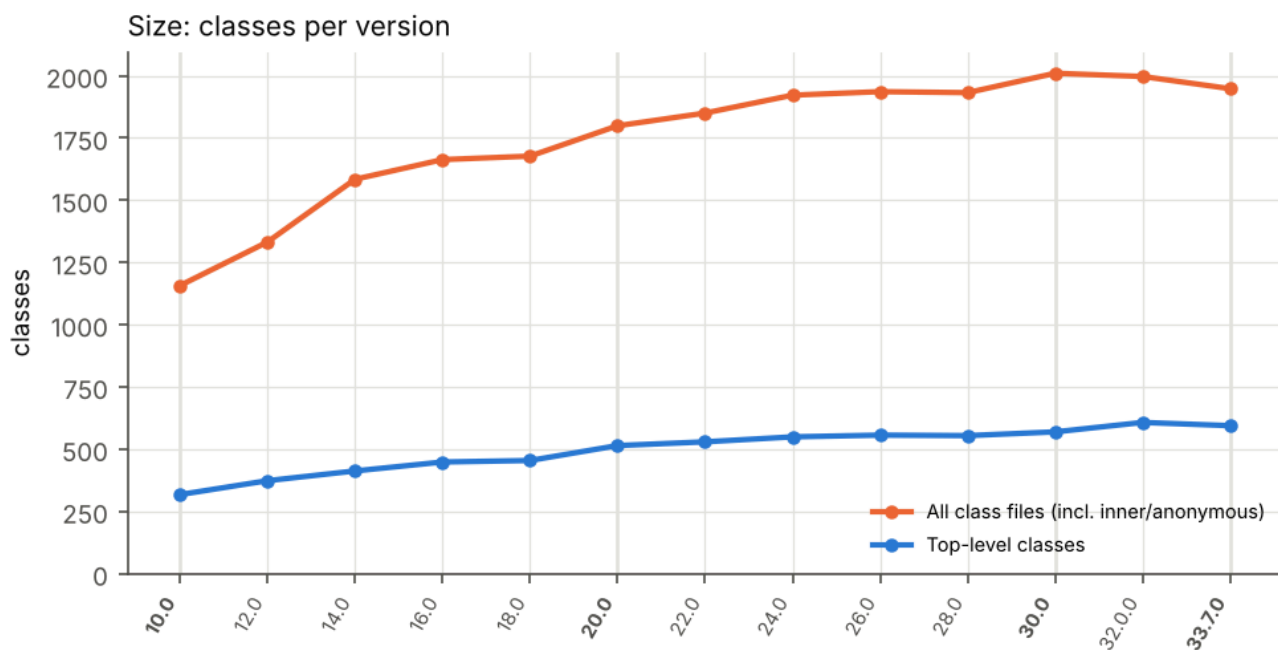


Figure 3 – Classes per version (versions on an ordinal axis).

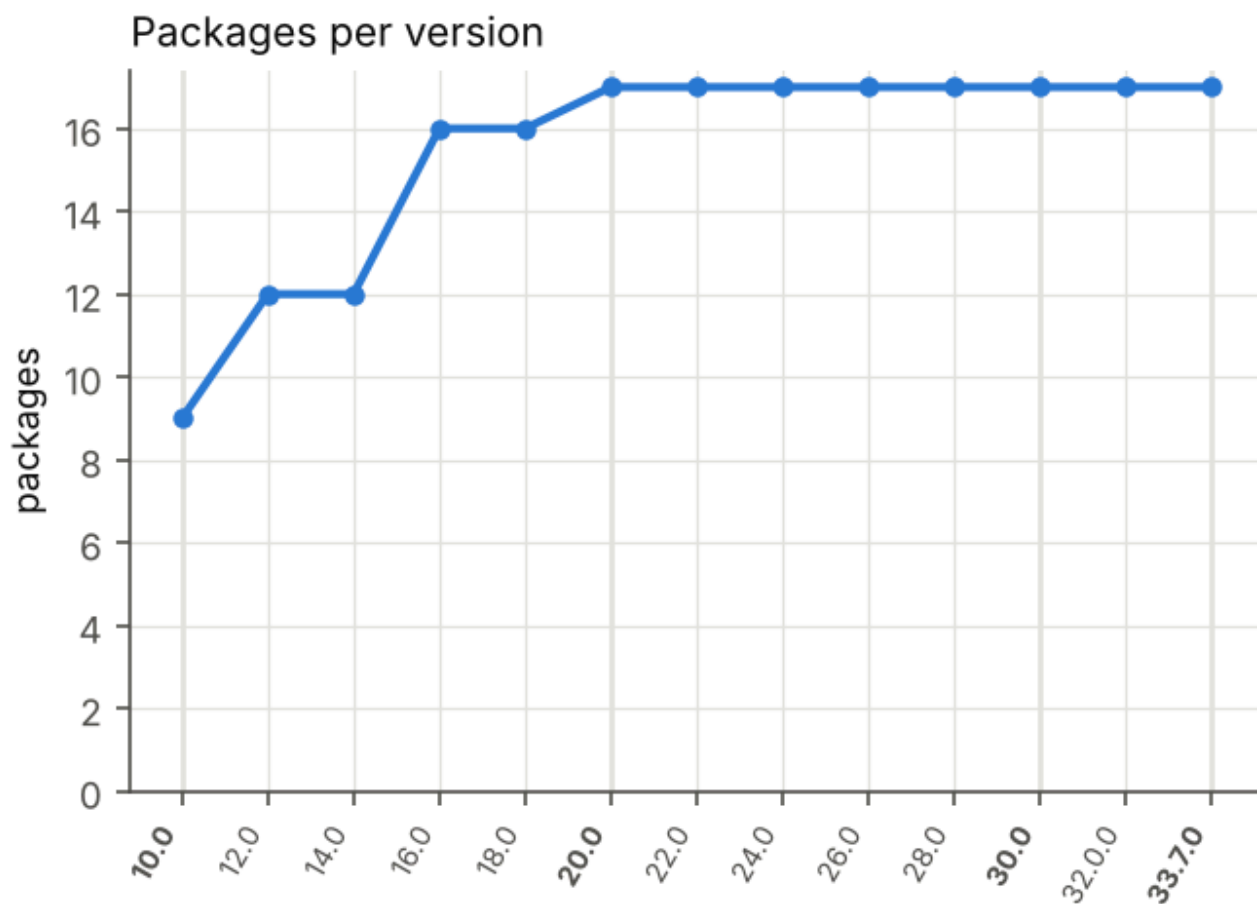


Figure 4 – Packages per version.

Guava grew from **317 to 594 top-level classes (+87 %)** and from 1,157 to 1,948 class files over 15 years. Growth is concentrated in the first five years:

- **10.0 → 20.0 (2011-2016): +62 %.** New packages were added: hash, math and reflect in 12.0; escape, html, xml and the public-suffix list in 16.0; and the graph library in 20.0 (+76 classes). The number of packages grew from 10 to 18.
- **20.0 → 32.0 (2016-2023): +18 %**, without new packages. The changes are additions to existing packages, mainly Java 8 support between 20.0 and 24.0 (Streams, MoreCollectors, CollectSpliterators, FluentFuture, MoreFiles, ImmutableIntArray) and graph traversal (Traverser).
- **32.0 → 33.7 (2023-2026): -2 %.** 14 classes were added and 18 removed (Table 9). The library is now in maintenance mode.

Lehman’s laws. *Continuing change (I)* holds: every sampled release changes classes (Table 9). *Continuing growth (VI)* holds, but with a strongly decreasing rate. This is the inverse-square growth that Turski’s model associates with the growing effort of changing a large system, and it is reinforced here by Guava’s compatibility policy: public API can only be removed after deprecation, and new API must fit an existing package. Since 20.0 no sampled release added more than 5 % new classes, which is consistent with *conservation of familiarity (V)*.

3.2 Class dependencies

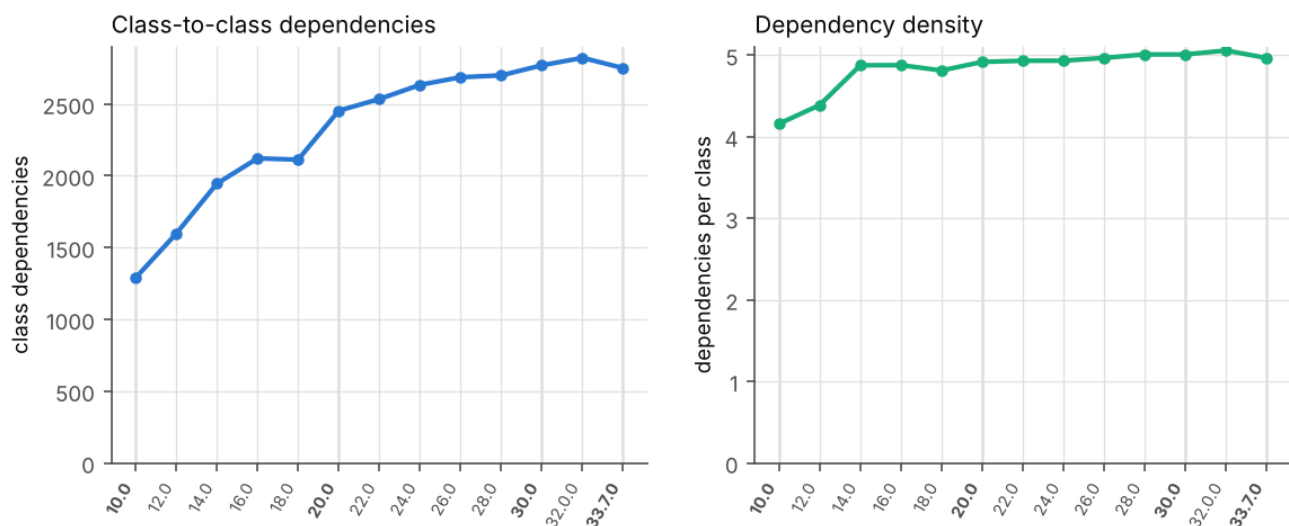


Figure 5 – Dependencies (left) and dependencies per class (right).

Dependencies grew from **1,289 to 2,750 (×2.1)**, faster than the classes (×1.8). The density rose from **4.2 to 5.1 dependencies per class** (peak in 32.0.0) and, unlike in a system that is actively restructured, it **never decreases**. This is a clear case of *increasing complexity (II)*. New features are built on top of the existing collection and base APIs, so each new class adds dependencies to a dense core. Guava is also denser than an application framework: our first system (Spring AI) had 2.9–4.1 dependencies per class.

3.3 Widely-used (noise) classes

Table 3 – Noise classes (JNode) per version; *Top-30 fan-in flagged* = how many of the 30 most-used classes JNode flags.

Version	Connected classes	Isolated	Noise (top-level)	Noise %	Top-30 fan-in flagged	Fallback
10.0	302	8	24	7.95%	12	yes
12.0	354	10	15	4.24%	5	yes
14.0	390	9	48	12.31%	7	
16.0	425	10	58	13.65%	11	
18.0	429	10	58	13.52%	10	
20.0	491	8	68	13.85%	13	
22.0	506	8	73	14.43%	13	
24.0	527	7	75	14.23%	13	
26.0	535	6	75	14.02%	13	
28.0	533	6	75	14.07%	13	
30.0	547	6	82	14.99%	16	
32.0.0	553	5	68	12.3%	11	
33.7.0	549	5	77	14.03%	11	

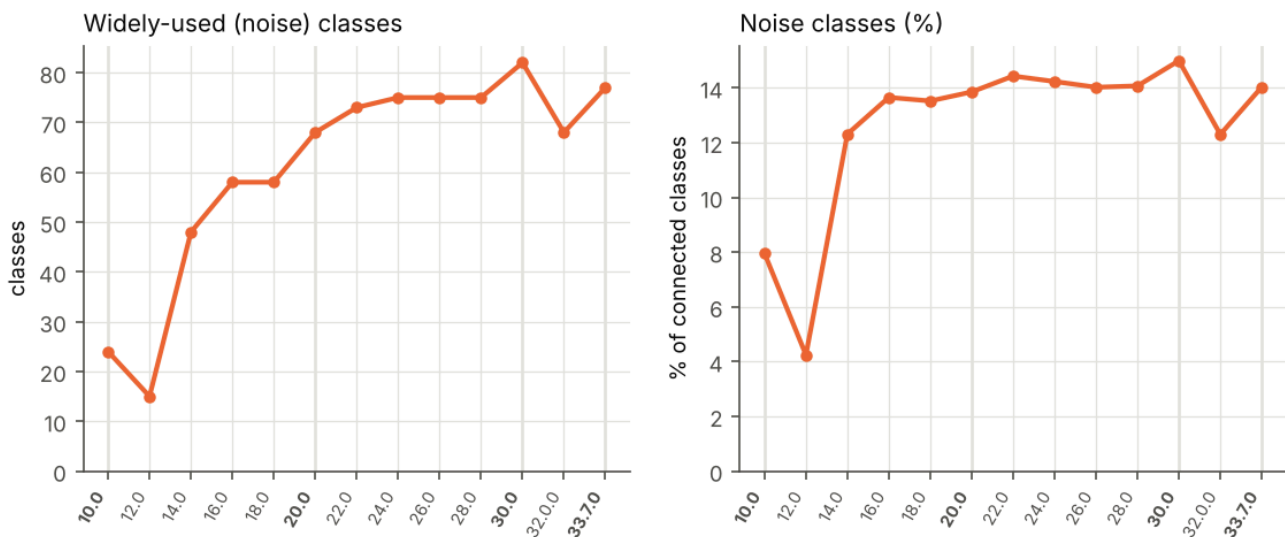


Figure 6 – Noise classes per version.

JNode flags **15-82 top-level classes (4-15 %)**. They have a fan-in about 4 times higher than the other classes (12-20 vs 3-4). But **what** JNode flags is only partly what one would call “omnipresent”:

- In 33.7.0, **43 of the 48 classes of `common.base`** are noise, which is plausible (Preconditions, Function, Predicate, Optional, CharMatcher, ...).
- **18 of the 19 classes of `common.cache` are also flagged**, although the cache is used by only one other package (eventbus, 3 dependencies). The cache depends on base, collect and concurrency, so it lies on many shortest paths inside its reachable subgraph and gets a high SIG. That makes it “significant”, not “widely used”.
- Many genuine hubs are **not** flagged: only 5-16 of the 30 classes with the highest fan-in are noise. Missed are Iterators, Ordering, Multiset, ImmutableCollection, Iterables, Lists, Sets, Multimap and hashCode, which are core collection types used throughout the library.

As in our first analysis, JNode’s SIG is a reachability measure, and its noise set must be sanity-checked against fan-in. For Guava, removing the noise mostly removes `common.base` and `common.cache` from the graph.

3.4 Recovered architectures (A1-A4)

Table 4 - Mean over the 13 versions.

Architecture	k	Largest cluster	Cohesion	Coupling	MQ (course)	TurboMQ	TurboMQ/k	Intra deps	ARI vs packages
A1 ACDC (full)	78	46	0.2578	0.00649	0.2513	24.1	0.314	0.368	0.106
A2 ACDC (no noise)	69	38	0.2506	0.00481	0.2458	29.8	0.433	0.470	0.101
A3 k-means (full)	60	31	0.1756	0.00662	0.1690	20.3	0.342	0.296	0.074
A4 k-means (no noise)	52	18	0.1865	0.00408	0.1824	26.2	0.506	0.441	0.083
Packages (full)	15	205	0.1053	0.00503	0.1003	7.2	0.500	0.714	1.000
Packages (no noise)	14	196	0.1161	0.00363	0.1125	8.8	0.652	0.885	1.000

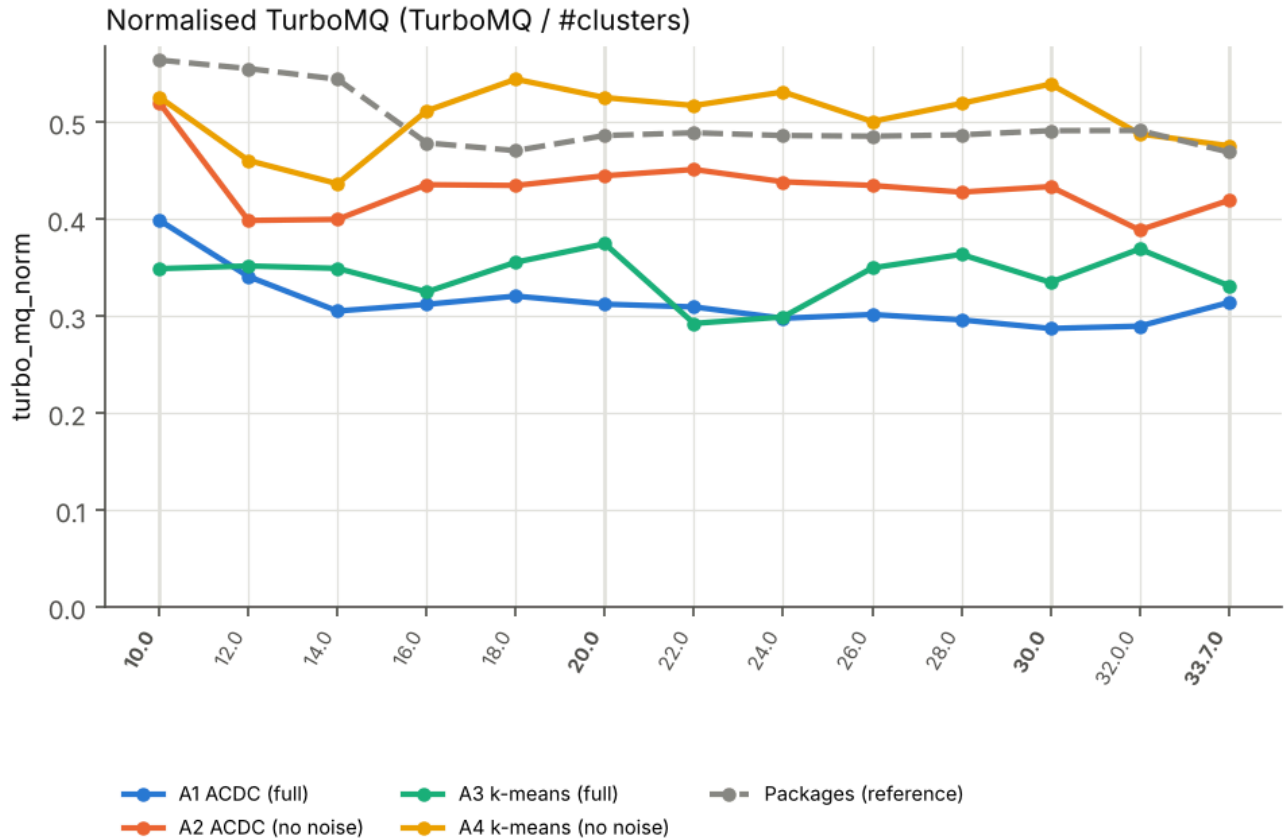


Figure 7 - Normalised TurboMQ (TurboMQ/k).

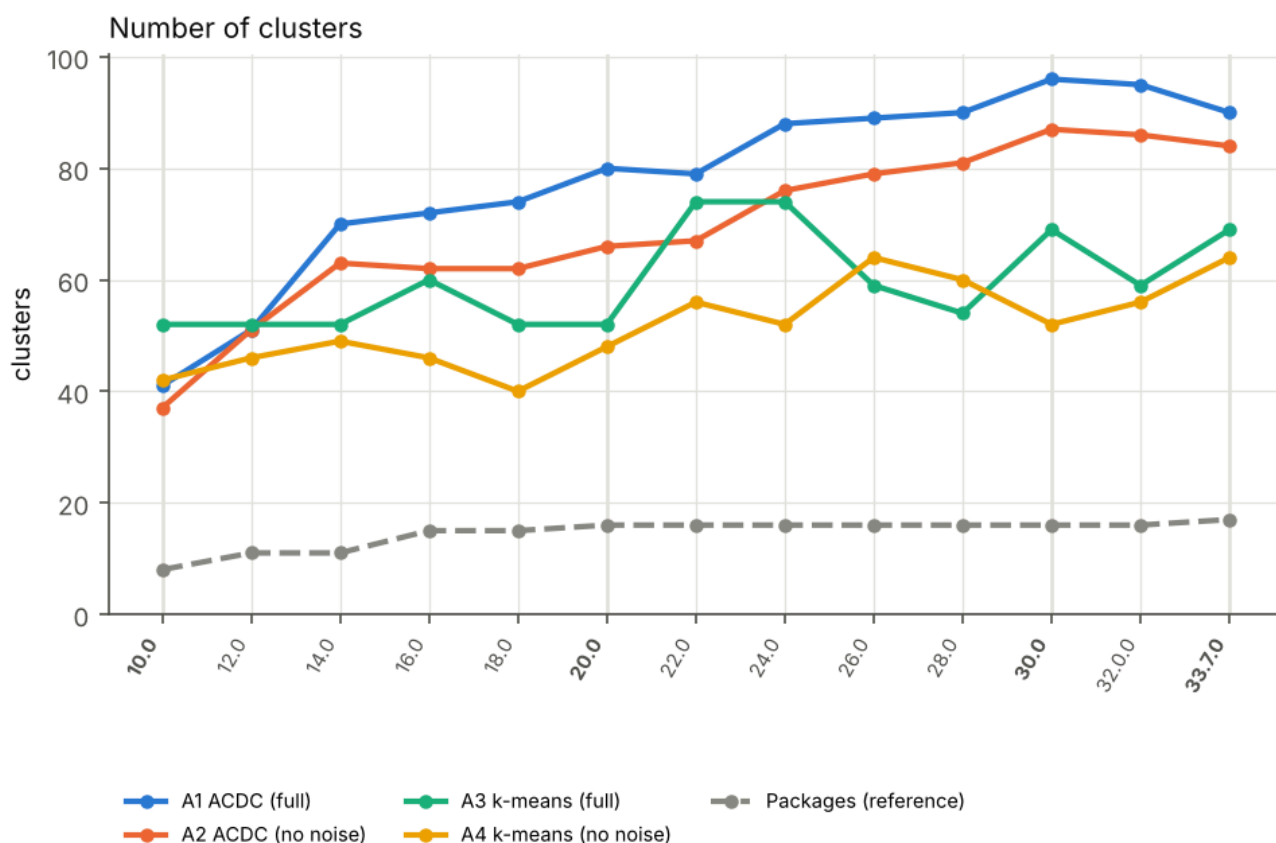


Figure 8 - Number of clusters.

Size. ACDC produces 41 (10.0) to 90-96 clusters (30.0-33.7.0), with a largest cluster of 27-52 classes. k-means produces 40-74 clusters of at most 43 classes. The packages are very coarse: 8-17 groups, the largest being `common.collect` with 179-217 classes.

Quality by the course MQ - ACDC wins, the packages are last. ACDC has the highest MQ in every version (0.24-0.26; mean 0.251 for A1, 0.246 for A2), k-means follows (0.169 / 0.182), and the package structure is clearly last (0.100 on average, between 0.084 and 0.121, lowest in 33.7.0). MQ is dominated by its cohesion term: ACDC's clusters of 5-7 classes are dense, while `common.collect` with ~200 classes can never be (μ_i/N_i^2).

Quality by the k-independent measures - the packages win. Guava is the opposite of an application framework. Its **package decomposition is by far the most modular on the dependency share: 71 % of all dependencies stay inside a package**, against 37 % (A1) and 30 % (A3). On TurboMQ/k the packages (0.50) are better than A1/A2/A3 and equal to A4. Guava's packages are cohesive feature libraries (`graph`, `hash`, `io`, `util.concurrent`) built around a dense core. The algorithms have no notion of "feature". They see one densely connected graph, and to reach clusters of 5-10 classes they must cut through the dense core (`collect`, `base`), which costs many dependencies. Their much higher **cohesion** (0.25 for ACDC vs 0.11 for packages) only reflects their small cluster size (μ_i/N_i^2), not a better modularisation.

- **ACDC vs k-means.** ACDC has the higher cohesion and MQ and keeps more dependencies inside clusters (37 % vs 30 % on the full graph). k-means has the higher TurboMQ/k (0.34 vs 0.31 on the full graph, 0.51 vs 0.43 without noise).
- **Noise removal** (A1→A2, A3→A4) raises TurboMQ/k by **+0.12 (ACDC)** and **+0.16 (k-means)** and the intra share by 10-15 percentage points. It also raises the packages'

TurboMQ/k from 0.50 to 0.65. Removing base cuts most edges that cross package borders. The course MQ hardly changes (ACDC 0.251 $\rightarrow$ 0.246, k-means 0.169 $\rightarrow$ 0.182).

ACDC parameters. On 33.7.0 (Table 6, Figure 9): BodyHeader has no effect (bso = so). Without OrphanAdoption, ClusterLast puts the orphans into one cluster of 276 classes, and TurboMQ/k falls from 0.31 to 0.25. The maximum cluster size of SubGraph has practically **no influence** (5 $\rightarrow$ 91 clusters, $\geq 10 \rightarrow 90$): Guava's dominator subgraphs are naturally small.

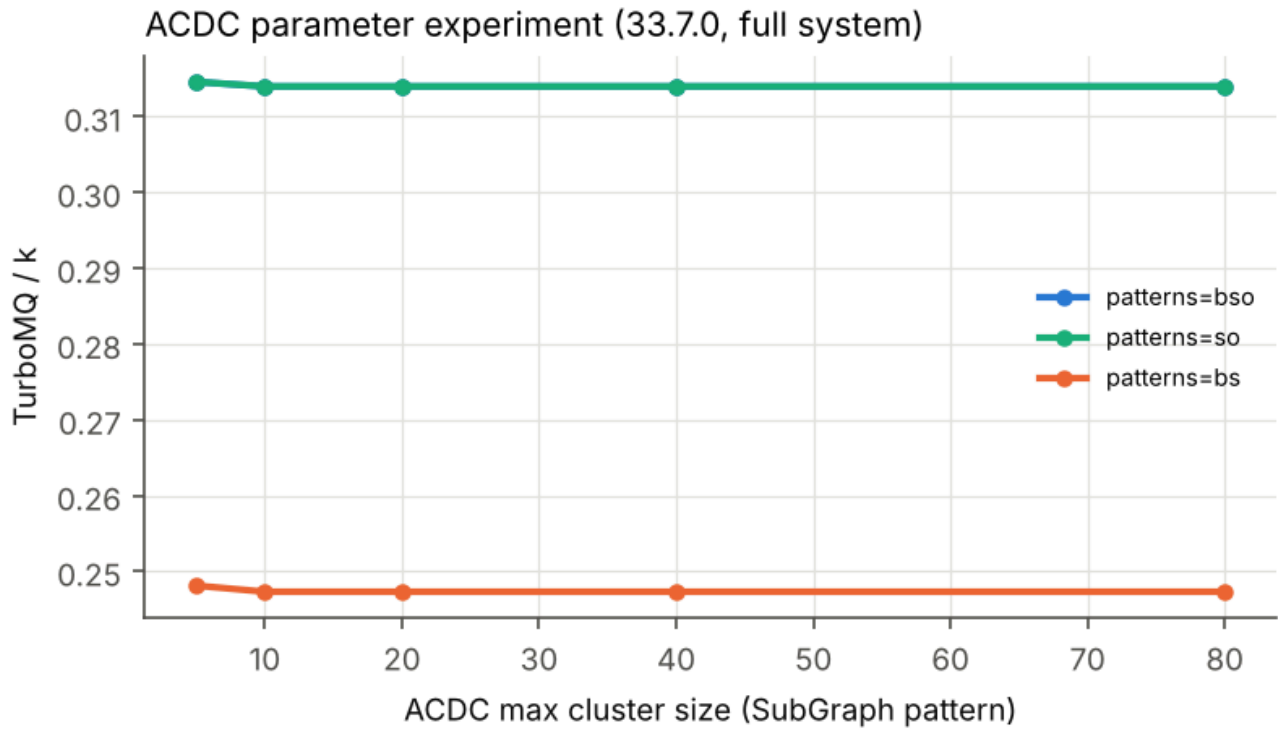


Figure 9 - ACDC parameter experiment (33.7.0, full graph); bso lies under so.

Table 6 - ACDC parameter experiment (33.7.0).

Graph	Patterns	Max size	k	Largest	TurboMQ/k	Intra deps
full	bso	5	91	47	0.3146	0.3589
full	bso	10	90	47	0.314	0.3593
full	bso	20	90	47	0.314	0.3593
full	bso	40	90	47	0.314	0.3593
full	bso	80	90	47	0.314	0.3593
full	so	5	91	47	0.3146	0.3589
full	so	10	90	47	0.314	0.3593
full	so	20	90	47	0.314	0.3593
full	so	40	90	47	0.314	0.3593
full	so	80	90	47	0.314	0.3593
full	bs	5	92	276	0.2481	0.4156
full	bs	10	91	276	0.2473	0.416
full	bs	20	91	276	0.2473	0.416
full	bs	40	91	276	0.2473	0.416
full	bs	80	91	276	0.2473	0.416
nonoise	bso	5	84	48	0.4193	0.4612
nonoise	bso	10	84	48	0.4193	0.4612
nonoise	bso	20	84	48	0.4193	0.4612
nonoise	bso	40	84	48	0.4193	0.4612
nonoise	bso	80	84	48	0.4193	0.4612
nonoise	so	5	84	48	0.4193	0.4612
nonoise	so	10	84	48	0.4193	0.4612
nonoise	so	20	84	48	0.4193	0.4612
nonoise	so	40	84	48	0.4193	0.4612
nonoise	so	80	84	48	0.4193	0.4612
nonoise	bs	5	85	219	0.3424	0.4353
nonoise	bs	10	85	219	0.3424	0.4353
nonoise	bs	20	85	219	0.3424	0.4353
nonoise	bs	40	85	219	0.3424	0.4353
nonoise	bs	80	85	219	0.3424	0.4353

3.5 Quality of the architecture over time

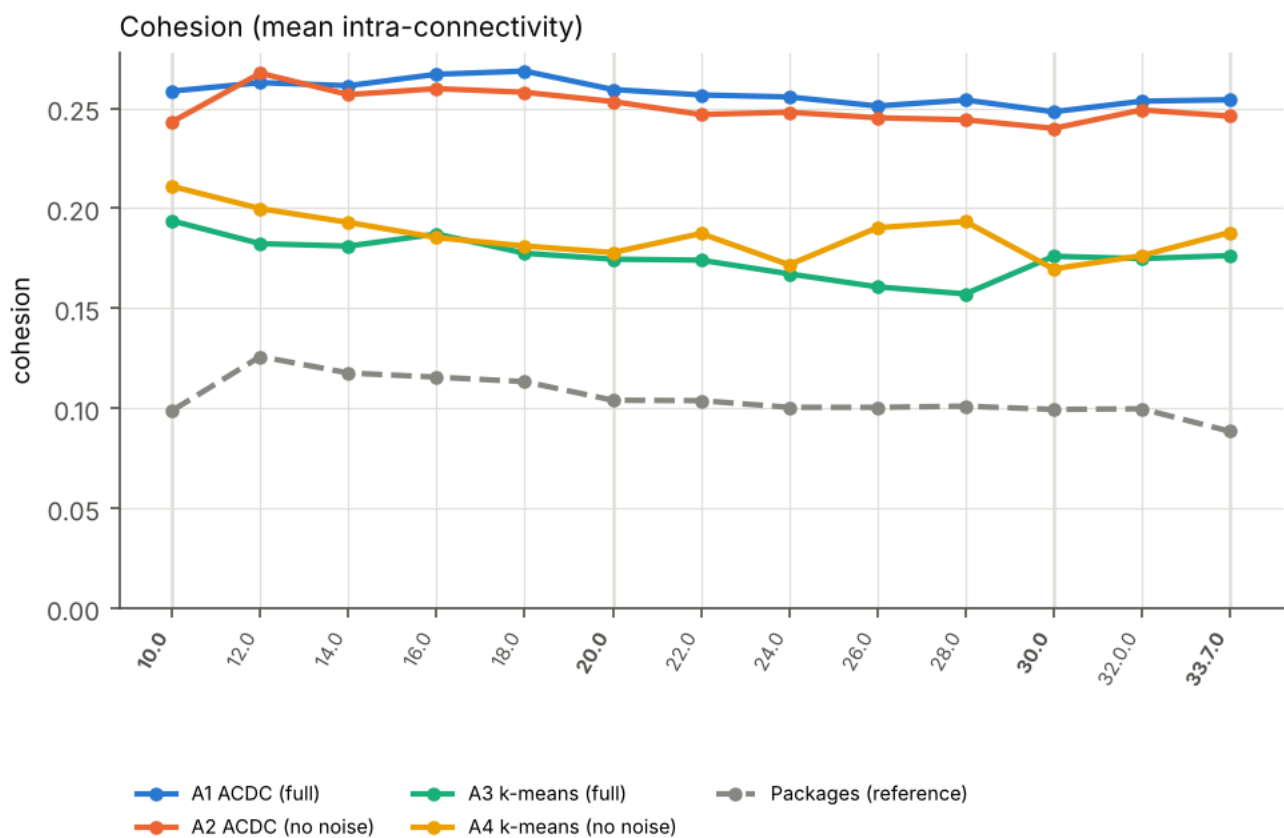


Figure 10 - Cohesion.

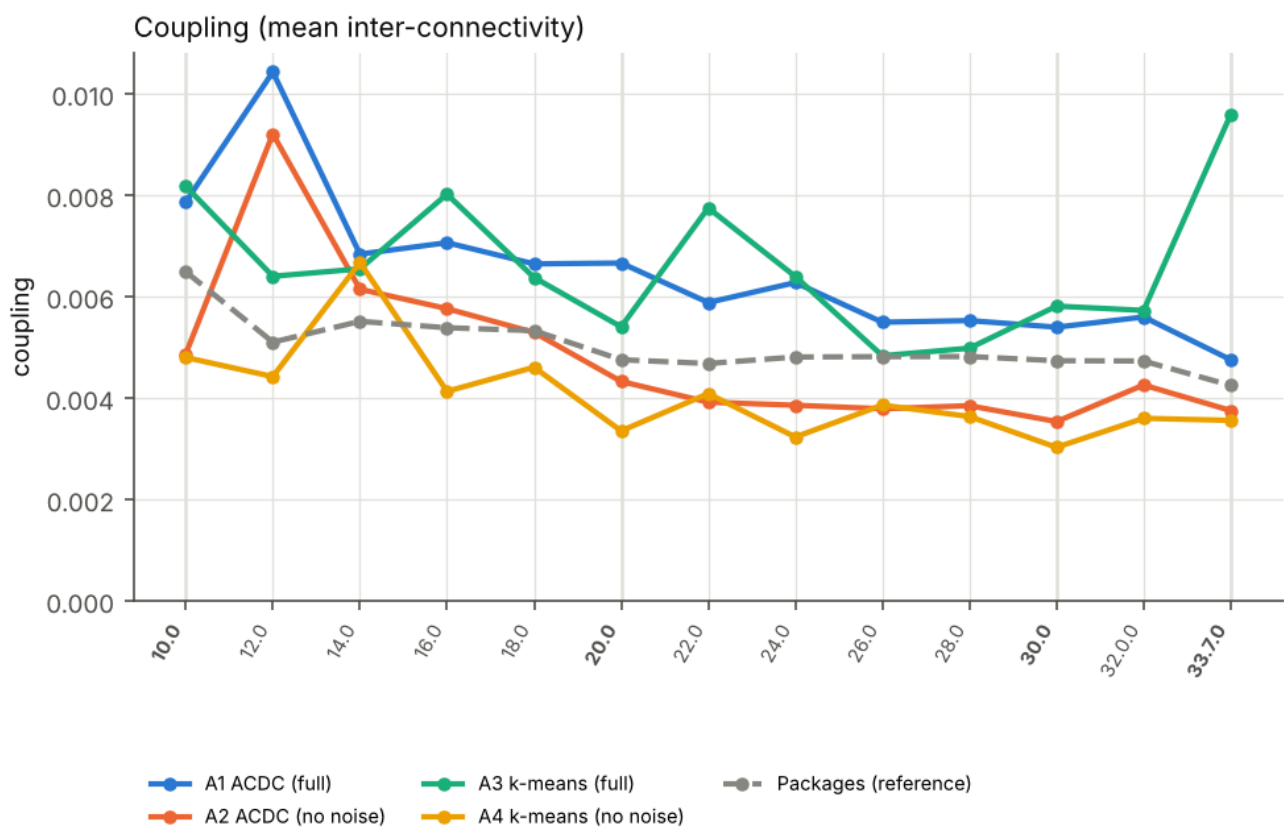


Figure 11 - Coupling.

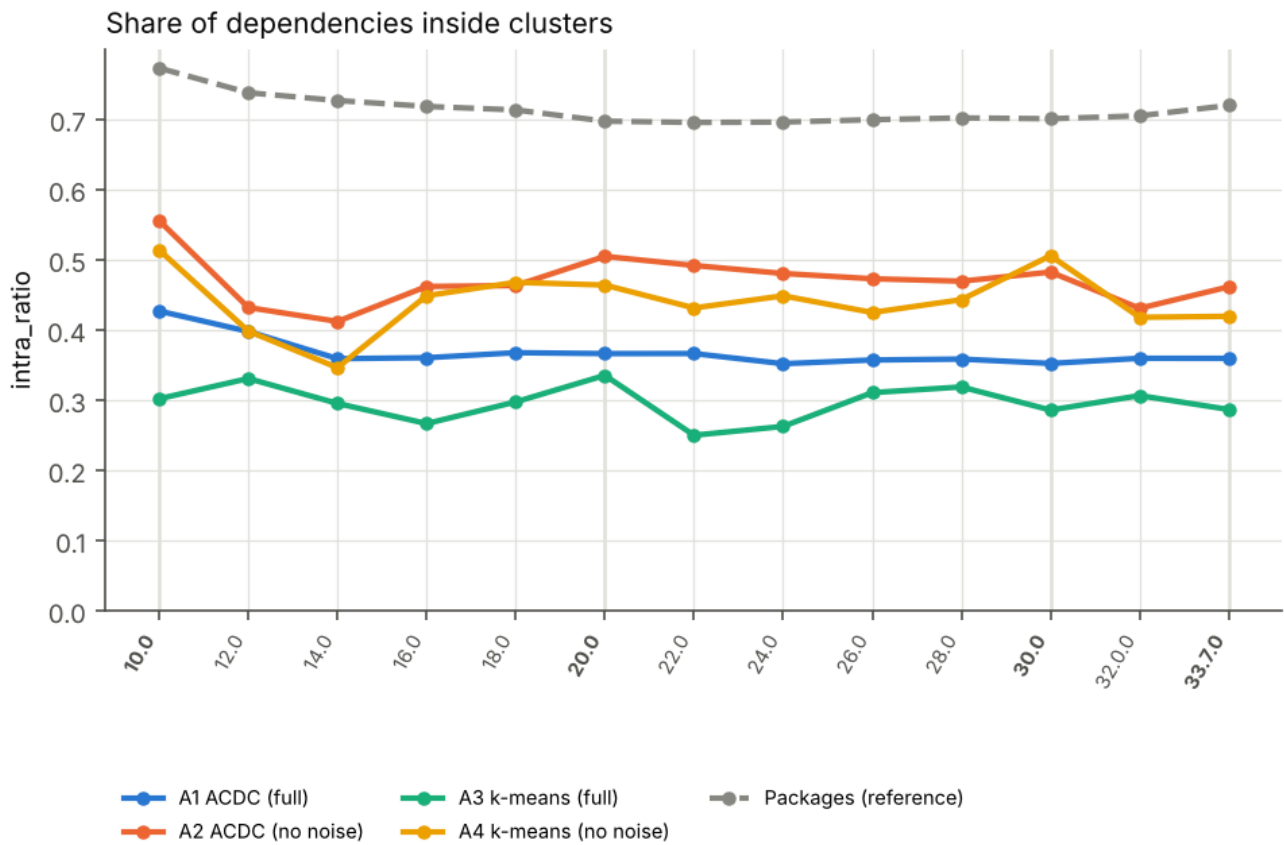


Figure 12 - Share of dependencies inside clusters.

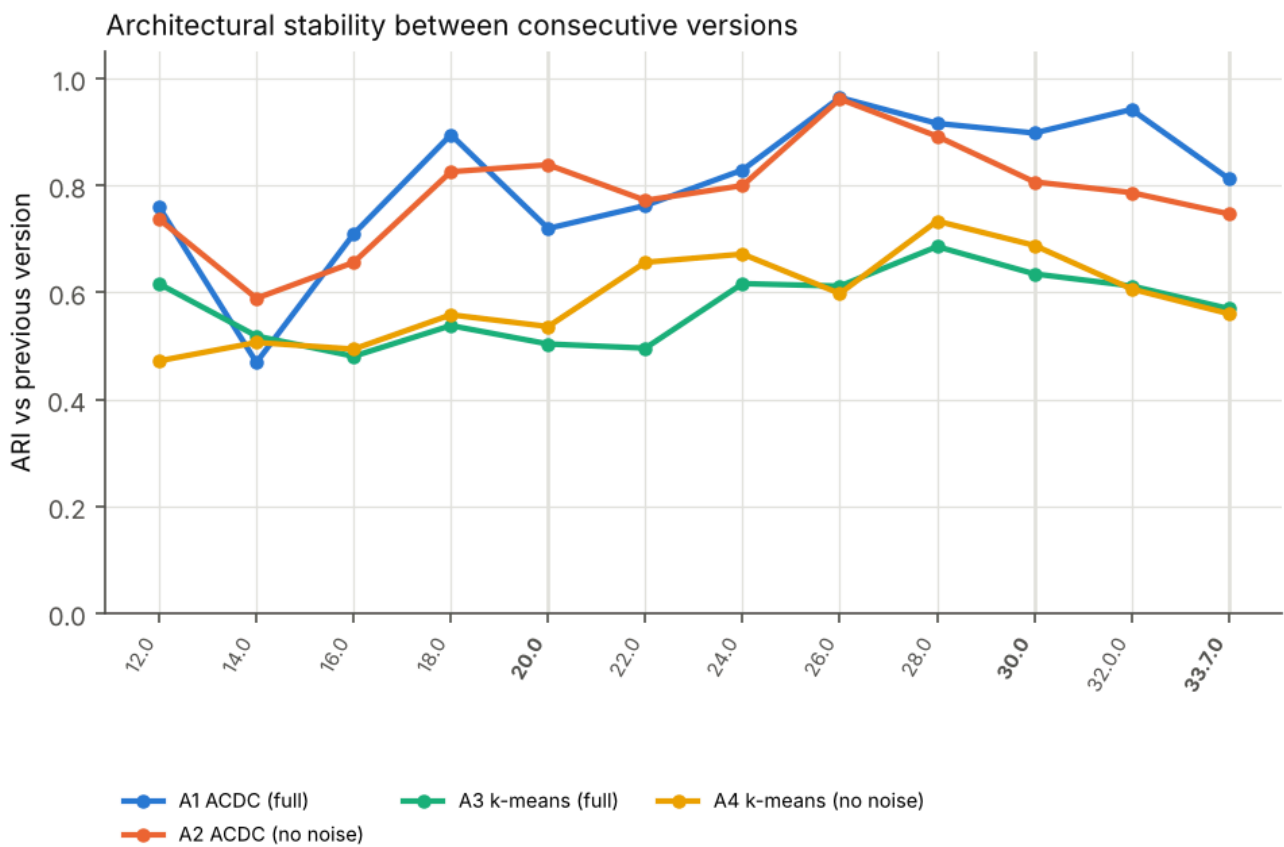


Figure 13 - Stability between consecutive sampled versions.

1. **10.0 → 16.0 - modularity declines while the library expands.** ACDC TurboMQ/k falls from 0.40 to 0.31, its intra share from 0.43 to 0.36, and the package decomposition from 0.56 to 0.48. New packages (hash, math, reflect, escape, ...) are not independent: they depend on base, collect and primitives. Each one adds cross-package edges.
2. **16.0 → 33.7.0 - stable at a lower level.** All full-graph measures stay flat: A1 TurboMQ/k 0.29–0.32, packages 0.47–0.49 with an intra share of 0.70–0.72. The graph library (20.0), the biggest addition of this period, is a well-separated package and does not reduce modularity.
3. **Coupling** (mean inter-connectivity) falls for ACDC (0.0079 → 0.0048) and the packages (0.0065 → 0.0042), but this mostly reflects the growing cluster count and size (ε_{ij} is divided by $N_i N_j$), not a real decoupling. The dependency density rises at the same time (3.2).

So Guava shows *declining quality (VII)* in its expansion phase and then stabilises. Unlike our first system, there is no restructuring phase that improves the measures again: the compatibility policy makes a re-modularisation of a public library practically impossible. Packages and public classes are API, and moving them would break users.

Stability. Between the sampled versions ACDC is more stable (mean ARI 0.81 for A1, 0.78 for A2) than k-means (0.57 / 0.59). Values are lower than for consecutive releases, because each step spans two majors (and up to 3 years). The least stable step is 12.0 → 14.0 (A1 ARI 0.47), where 58 classes were added and 23 removed.

3.6 Distance to the reference architecture (MoJoFM)

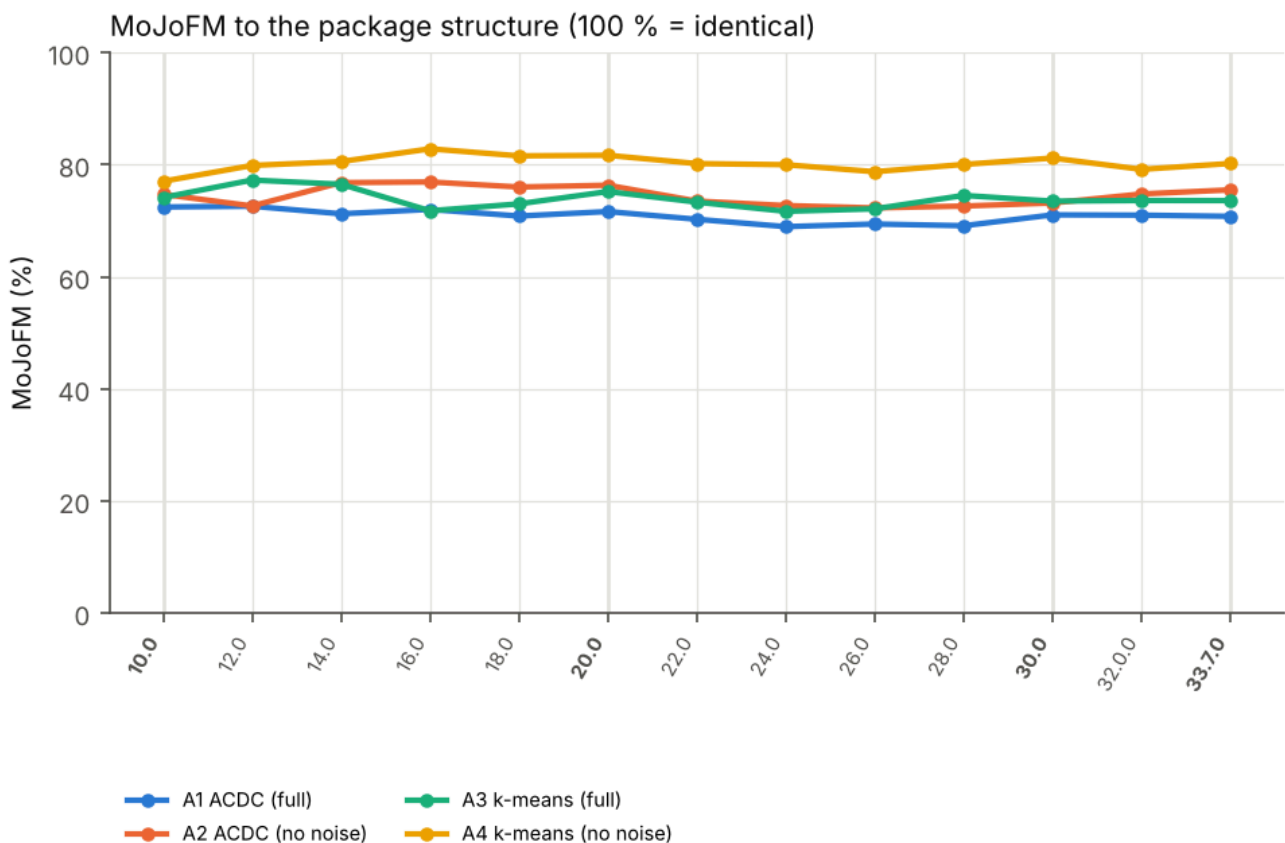


Figure 14 – MoJoFM of A1–A4 to the package structure.

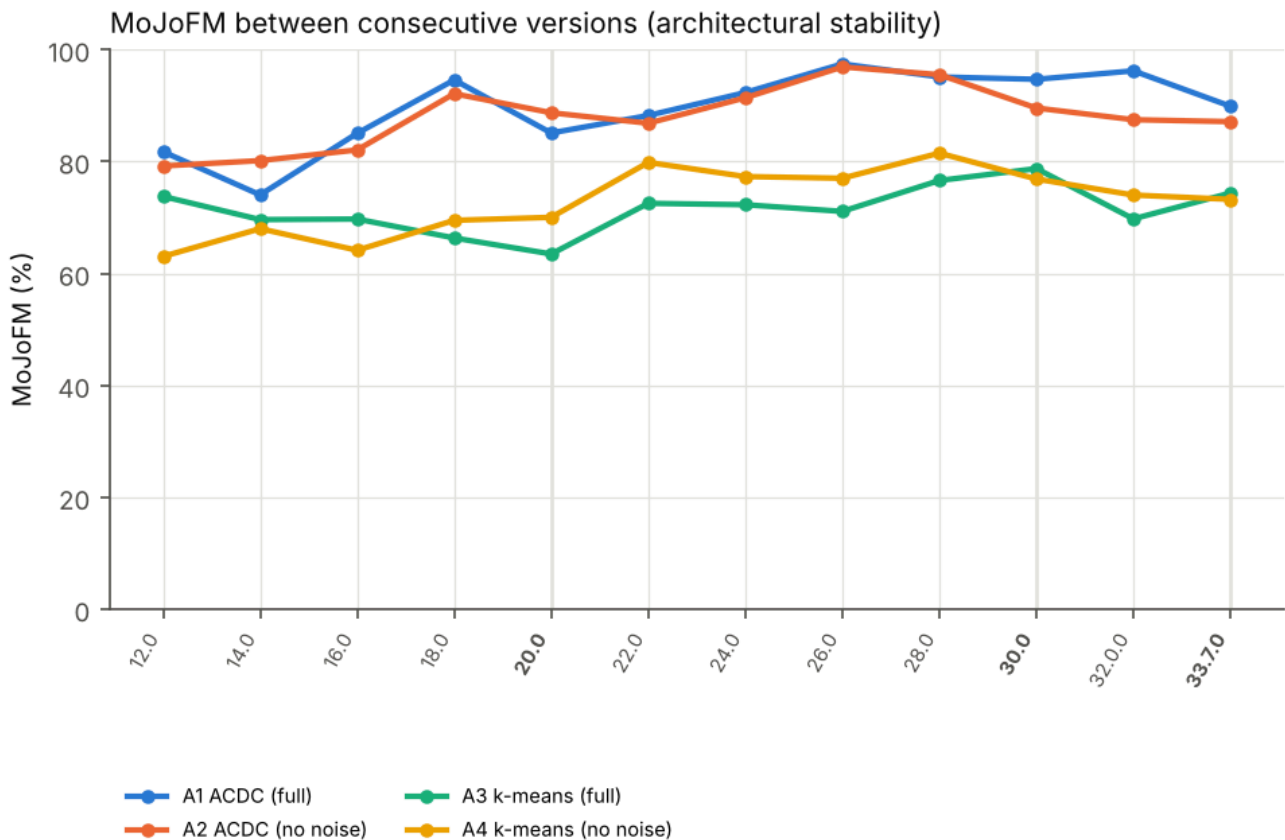


Figure 15 - MoJoFM between consecutive sampled versions.

Guava has no expert-validated architecture, so we use the package structure (all versions) and the AI architecture (33.7.0) as reference architectures for the lecture's MoJoFM.

- **To the packages** (Table 13): all recovered architectures are 69–83 % similar to the packages. Values are high because the packages are coarse: merging small clusters into them needs few Move/Join operations. Unlike in Spring AI, **k-means is closer than ACDC** (A3 73.9 % vs A1 70.9 %), and **removing the noise classes brings both closer** (A2 74.5 %, A4 80.2 %), as in the lecture's experiment.
- **To the AI architecture** (Table 14): the packages are closest (75.7 %), then k-means (69.0 %, 74.6 % without noise), then ACDC (66.0 %, 68.7 %). Conversely, the AI architecture is 94.4 % similar to the packages. It differs mainly by splitting `collect` and by grouping small packages (`escape/html/xml/net`, `primitives/math`).
- **Stability** (Table 15): ACDC is much more stable than k-means (mean MoJoFM 89.5 % / 88.1 % vs 71.5 % / 72.9 %). ACDC's stability grows as the library matures: 74–85 % in the expansion steps (12.0–16.0 and 20.0), 92–97 % between 24.0 and 32.0.

Table 13 - MoJoFM (%) of the recovered architectures to the package structure.

Version	A1 → PKG	A2 → PKG	A3 → PKG	A4 → PKG
10.0	72.45	74.71	74.15	77.04
12.0	72.59	72.64	77.26	79.87
14.0	71.24	76.8	76.52	80.56
16.0	72.02	76.92	71.78	82.84
18.0	70.84	76.02	73.01	81.58
20.0	71.64	76.34	75.21	81.68
22.0	70.26	73.51	73.32	80.2
24.0	68.95	72.71	71.68	80.0
26.0	69.42	72.29	72.12	78.75
28.0	69.11	72.62	74.52	80.05
30.0	71.05	73.17	73.5	81.19
32.0.0	71.0	74.78	73.61	79.17
33.7.0	70.79	75.51	73.6	80.22

Table 14 – MoJoFM (%) to the AI architecture (33.7.0).

MoJoFM (%) to the AI architecture	A1	A2	A3	A4	Packages
33.7.0	65.98	68.69	68.97	74.55	75.7

Table 15 – MoJoFM (%) between consecutive sampled versions.

Version (vs previous)	A1	A2	A3	A4
12.0	81.75	79.2	73.76	63.04
14.0	74.04	80.16	69.62	68.0
16.0	85.04	82.07	69.72	64.14
18.0	94.57	92.12	66.34	69.49
20.0	85.1	88.71	63.48	70.03
22.0	88.22	86.84	72.53	79.84
24.0	92.32	91.39	72.28	77.31
26.0	97.44	96.9	71.09	77.01
28.0	95.11	95.52	76.61	81.5
30.0	94.69	89.56	78.74	76.87
32.0.0	96.2	87.47	69.75	74.01
33.7.0	89.88	87.12	74.37	73.24

3.7 Omnipresent classes: JNode vs Bunch

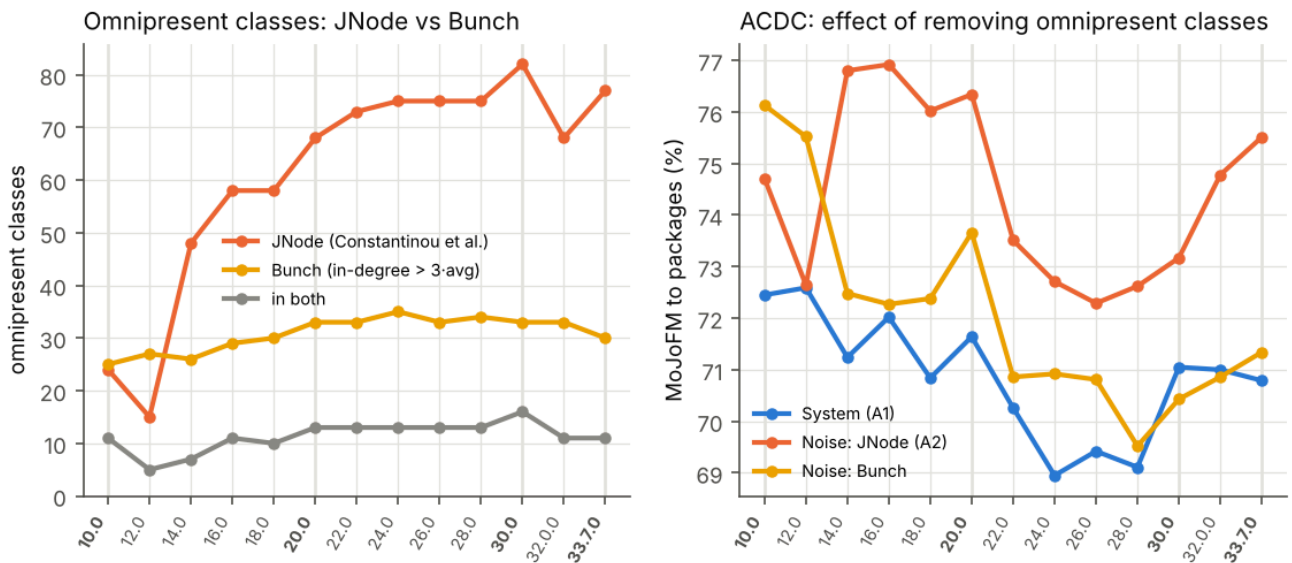


Figure 16 – Omnipresent classes by JNode and by the Bunch rule (left); MoJoFM of ACDC to the packages for the full system, without JNode noise and without Bunch omnipresent classes (right).

The Bunch rule (in-degree > 3 × average) flags **25-35 classes**, exactly the hubs that JNode misses (3.3). In 33.7.0 these are Preconditions, Function, Predicate, Supplier, MoreObjects, ImmutableList/Set/Map, Iterators, Iterables, Lists, Maps, Sets, Ordering, Multimap, Multiset, hashCode, Ints, ListenableFuture, EndpointPair, and others. The two sets hardly overlap (**Jaccard 0.10-0.29**): JNode flags most of base and cache, Bunch the most-used classes of collect and base.

For the effect on ACDC (Table 16), **JNode's noise removal brings ACDC closer to the packages from 14.0 on** (72-77 % vs 69-74 % for Bunch). Bunch is better only in 10.0 and 12.0, where JNode needed the threshold fallback. Removing the Bunch hubs cuts through collect and scatters its classes, while removing cache (JNode) removes a separate package that does not affect the other clusters. The course MQ stays at 0.23-0.26 for all variants.

Table 16 – JNode vs Bunch omnipresent classes and their effect on ACDC.

Version	Bunch (>3·avg)	JNode	Both	Jaccard	MQ System	MQ JNode	MQ Bunch	MoJoFM System	MoJoFM JNode	MoJoFM Bunch
10.0	25	24	11	0.289	0.251	0.238	0.244	72.45	74.71	76.13
12.0	27	15	5	0.135	0.252	0.258	0.250	72.59	72.64	75.52
14.0	26	48	7	0.104	0.254	0.251	0.233	71.24	76.8	72.48
16.0	29	58	11	0.145	0.260	0.254	0.245	72.02	76.92	72.27
18.0	30	58	10	0.128	0.262	0.253	0.249	70.84	76.02	72.38
20.0	33	68	13	0.148	0.253	0.249	0.244	71.64	76.34	73.66
22.0	33	73	13	0.14	0.251	0.243	0.238	70.26	73.51	70.86
24.0	35	75	13	0.134	0.249	0.244	0.235	68.95	72.71	70.92
26.0	33	75	13	0.137	0.246	0.241	0.231	69.42	72.29	70.81
28.0	34	75	13	0.135	0.249	0.240	0.236	69.11	72.62	69.52
30.0	33	82	16	0.162	0.243	0.236	0.233	71.05	73.17	70.43
32.0.0	33	68	11	0.122	0.248	0.245	0.235	71.0	74.78	70.86
33.7.0	30	77	11	0.115	0.249	0.242	0.238	70.79	75.51	71.34

3.8 Architectural smells and their evolution

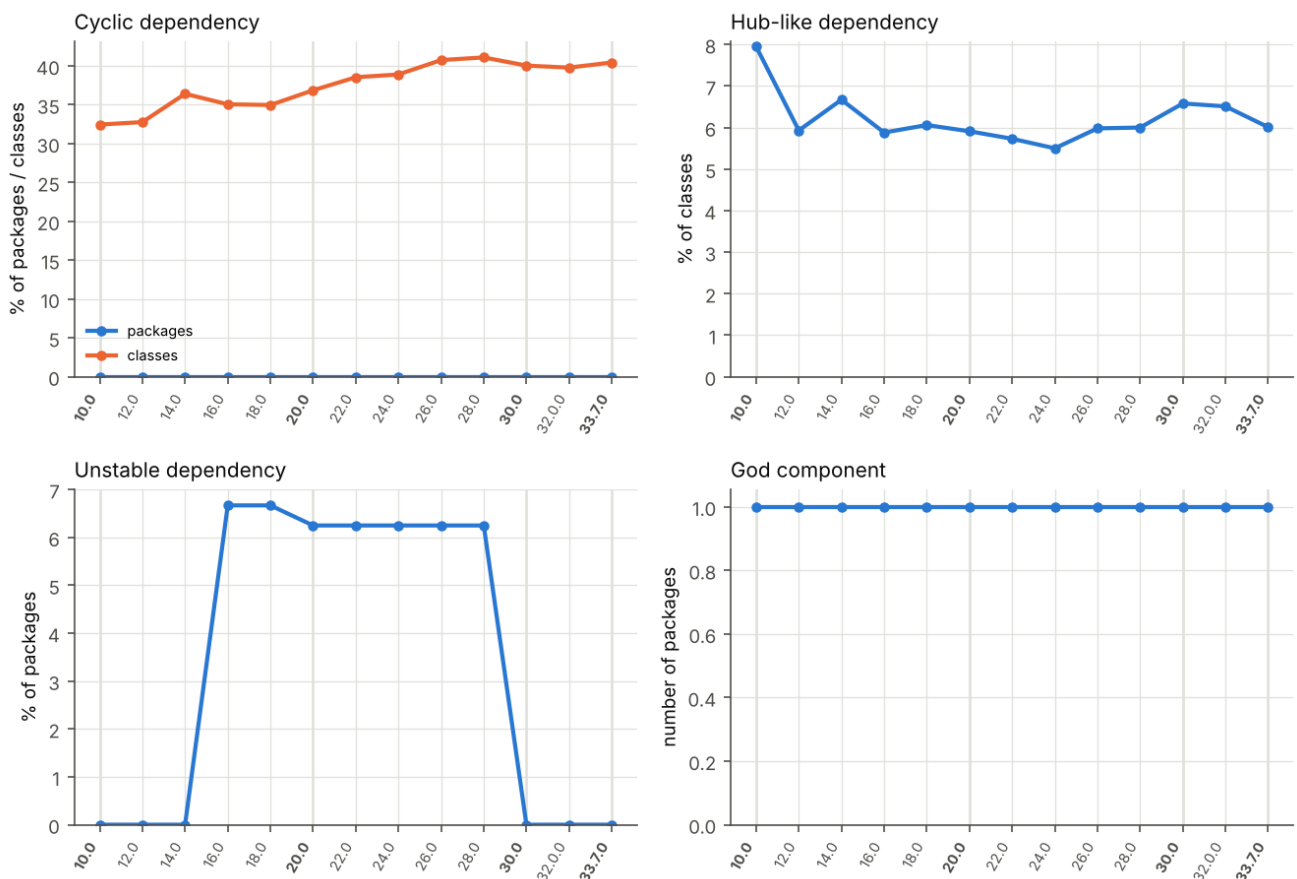


Figure 17 – Evolution of the architectural smells.

- **Cyclic dependency - none between packages, many inside them.** Guava's package dependency graph is **acyclic in all 13 versions**. This is a deliberate layering (base → primitives → collect → concurrency → features) that the maintainers have kept for 15 years. Inside the packages, however, the share of classes in a class-level cycle rises steadily from **32 % to 41 %**, almost all of them in `collect`, where interfaces, implementations and utilities reference each other (`ImmutableList` $\rightleftharpoons$ `ImmutableCollection` $\rightleftharpoons$ `Iterators`, ...). The package boundaries are kept clean, and the entanglement grows inside them.
- **Hub-like dependency** affects 5.5–8 % of the classes, highest in 10.0 and then stable at $\approx$ 6 %.
- **Unstable dependency:** only one package, `collect`, between 16.0 and 28.0. One of its three package dependencies, `math`, was less stable than `collect` itself (instability 0.29–0.33 vs 0.27–0.30). In 30.0 `math` became more stable than `collect`, and the smell disappears.
- **God component:** `common.collect` in **every** version (threshold 130–146 classes; `collect` has 179–217). It is the main architectural smell of Guava, and it is permanent because its classes are public API.

Table 17 – Architectural smells per version.

Version	Cyclic pkgs %	Cyclic classes %	Hub-like %	Unstable dep. %	God comp.	God components
10.0	0.0	32.45	7.95	0.0	1	common.collect
12.0	0.0	32.77	5.93	0.0	1	common.collect
14.0	0.0	36.41	6.67	0.0	1	common.collect
16.0	0.0	35.06	5.88	6.67	1	common.collect
18.0	0.0	34.97	6.06	6.67	1	common.collect
20.0	0.0	36.86	5.91	6.25	1	common.collect
22.0	0.0	38.54	5.73	6.25	1	common.collect
24.0	0.0	38.9	5.5	6.25	1	common.collect
26.0	0.0	40.75	5.98	6.25	1	common.collect
28.0	0.0	41.09	6.0	6.25	1	common.collect
30.0	0.0	40.04	6.58	0.0	1	common.collect
32.0.0	0.0	39.78	6.51	0.0	1	common.collect
33.7.0	0.0	40.44	6.01	0.0	1	common.collect

3.9 Lehman's laws: dependency changes and work rate

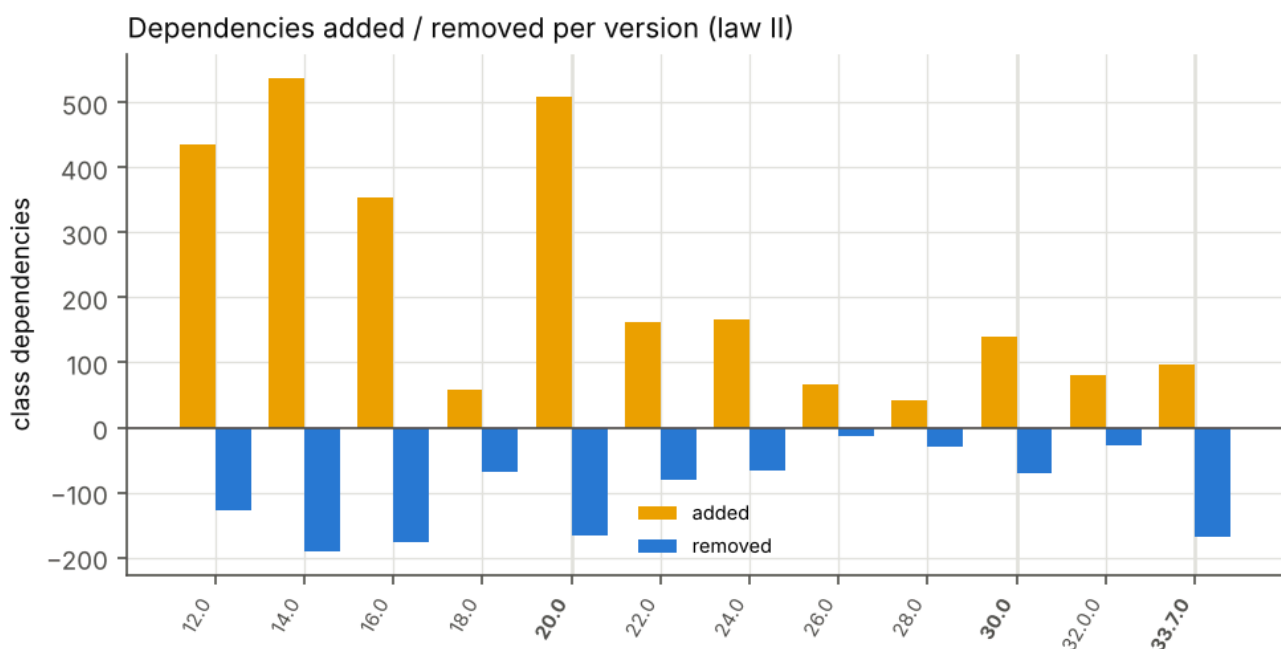


Figure 18 - Dependencies added and removed between sampled versions.

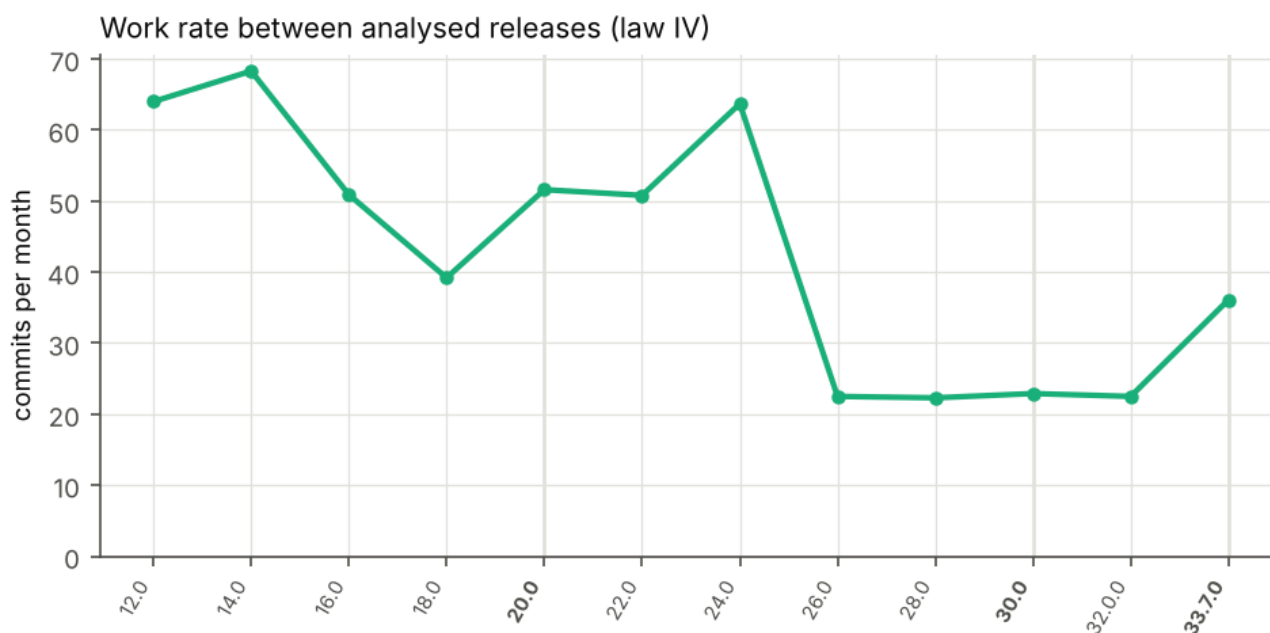


Figure 19 – Commits per month between the sampled releases.

- **Law II - increasing complexity.** In 10 of the 12 intervals Guava **adds more dependencies than it removes** (e.g. +538/−190 in 14.0, +508/−167 in 20.0). Only 18.0 (+59/−68) and 33.7.0 (+97/−168) are small net reductions. Unlike Spring AI there are no big restructuring releases, so complexity grows almost unchecked, matching the rising dependency density (3.2).
- **Law IV - conservation of organisational stability does not hold over 15 years.** The work rate falls from 64–68 commits/month (2011–2013) to 39–64 (2013–2018) and ≈ 22 (2018–2023), then rises again to 36 (2023–2026). This fits a maturing library whose work shifts to maintenance. (Caveat: until 18.0 Guava was mirrored from Google’s internal repository, so only 4–7 author identities appear in that period.)
- **Law VIII - feedback system.** Feedback reaches Guava through issues, @Beta APIs that may change after user feedback, deprecation cycles before removal, and the parallel JRE/Android flavours.
- Guava is an **E-type** system: it has to follow the Java platform (Java 8 support between 20.0 and 24.0, JDK changes), so the laws apply. Its strong compatibility promise is the reason laws II and VII show up so clearly: public structure cannot be refactored away.

Table 18 – Dependencies added and removed between sampled versions.

Version	Dependencies	Added	Removed
12.0	1597	436	128
14.0	1945	538	190
16.0	2121	353	177
18.0	2112	59	68
20.0	2453	508	167
22.0	2534	162	81
24.0	2633	166	67
26.0	2686	67	14
28.0	2699	42	29
30.0	2769	140	70
32.0.0	2821	80	28
33.7.0	2750	97	168

4. Architecture recovery with AI (latest version only: 33.7.0)

Tool and protocol. Claude (Anthropic) through Claude Code, with four prompts of increasing context (guava/ai/prompts.md, inputs in guava/ai/inputs/, verbatim responses in guava/ai/responses/).

Prompt	Information given
P1	Name, version and URL only
P2	README of 33.7.0 + the 17 packages with class counts and sample class names
P3	P2 + the complete list of 554 classes per package + the package dependency graph (50 weighted edges from DependencyExtractor). Because <code>common.collect</code> holds 38 % of the classes, the AI had to map every class (not only every package) to exactly one component, as ordered regex rules in JSON
P4	Follow-up with the metrics of the P3 architecture vs ACDC, k-means and packages

Limitation: the same assistant also built the analysis pipeline and had seen the data, so P1 is not a fully “cold” answer. Repeating the prompts with another tool (e.g. ChatGPT) would give an independent comparison.

What the AI produced.

- **P1 (no context):** a correct package-level layered view (base → primitives/math → collect → concurrency → feature libraries) with the right patterns (builders, immutable objects, forwarding decorators, template methods). Unlike for Spring AI, it contained **no outdated or invented parts**: Guava is old, stable and very well known, so the model’s general knowledge matches the current code. It was uncertain only about removed legacy APIs, correctly.
- **P2 (README + packages):** 9 components in 4 layers, and the main smell: `common.collect` as a “god package” (209 of 554 classes) whose encapsulation relies on

package-private visibility.

- **P3 (all classes + dependency graph):** 14 components. collect is split into *immutable collections*, *multi-collections*, *ranges* and *collection utilities*, with a measured component dependency table. It showed that **all dependency cycles are inside the collections subsystem** and that the rest is a clean layering. It also named the hub classes (Preconditions, ImmutableList, Maps, Iterators, ...).
- **P4 (metrics feedback):** explained why the split of collect lowers the share of internal dependencies below the packages, and proposed a hierarchical view. Merging the four collection components back into one gives 11 components with TurboMQ/k 0.624 and 72.5 % internal dependencies, better than the packages on both measures.

Table 10 – Components of the AI architecture (P3).

Component	Classes
Collection utilities, views & ordering	85
Concurrency (futures, services, executors)	78
Multi-collections (Multimap, Multiset, BiMap, Table)	61
Graph library	52
Immutable collections	48
Base utilities	47
Primitives & math	37
I/O	34
Hashing	31
Escaping, networking & public suffix	21
Caching	19
Ranges & discrete domains	15
Reflection	13
Event bus	8

Table 11 – AI (P3) architecture vs the other architectures of 33.7.0.

Architecture (33.7.0)	k	Largest	Cohesion	Coupling	MQ (course)	TurboMQ	TurboMQ/k	Intra deps
AI (P3), full	14	85	0.09365	0.003724	0.08993	8.042	0.5744	0.5578
AI (P3), no noise	13	81	0.08532	0.001874	0.08344	8.918	0.686	0.701
A1 ACDC	90	47	0.25423	0.004754	0.24947	28.26	0.314	0.3593
A2 ACDC–noise	84	48	0.24606	0.003752	0.24231	35.221	0.4193	0.4612
A3 k-means	69	40	0.17635	0.009588	0.16676	22.861	0.3313	0.2865
A4 k-means–noise	64	16	0.1879	0.003558	0.18434	30.44	0.4756	0.4192
Packages	17	209	0.08848	0.004248	0.08423	7.974	0.4691	0.7196

Table 12 – Agreement (ARI) of the AI architecture with the other architectures.

	A1	A2	A3	A4	Packages
AI (full)	0.2036	0.2127	0.1706	0.1841	0.5843
AI (no noise)	0.2231	0.2127	0.1835	0.1841	0.5312

5. Reflection

5.1 Comparing the architectures of the latest version

- **Granularity.** AI: 14 components; packages: 17; k-means: 64–69; ACDC: 84–90. The AI and the packages are at the level of a component diagram. The algorithmic architectures are fine-grained “class neighbourhoods”.
- **Quality - the verdict depends on the measure.** By the **course MQ**, ACDC is best (0.249), then k-means (0.167/0.184), and the coarse decompositions are last: AI 0.090 and packages 0.084, because MQ’s cohesion divides by N_i^2 . On the k-independent measures the AI architecture has the **best TurboMQ/k of all (0.574; 0.686 without noise)**, ahead of the packages (0.469), k-means (0.33/0.48) and ACDC (0.31/0.42). On the share of internal dependencies the packages are best (72 %), then the AI (56 %), then ACDC (36 %) and k-means (29 %). The AI pays for splitting collect into four parts. Merged, it beats the packages on both measures (P4).
- **Agreement.** The AI architecture agrees **moderately with the packages (ARI 0.58)** and weakly with ACDC (0.20) and k-means (0.17). ACDC and k-means agree with each other at ARI 0.26 (full) / 0.31 (no noise), and with the packages at only ≈ 0.1 . For a library organised by feature, the developers’ packages and an architect’s view largely coincide, while connectivity-based clustering finds a different, cross-cutting structure. With the lecture’s MoJoFM and the AI architecture as reference, the packages are closest (76 %), then k-means (69–75 %) and ACDC (66–69 %). In the other direction the AI architecture is 94 % similar to the packages (3.6).
- **Different principles.** The algorithms group classes that *use the same things*. For example, ACDC’s `Multimaps.ss` cluster gathers the multimap implementations and their utilities, and `Hashing.ss` the hash functions. These are sensible “class families”, but they ignore the feature boundaries that Guava’s users and maintainers work with.

Our assessment. For Guava the **AI architecture (P3, possibly with the collections merged as in P4) is the most useful and the best by the k-independent metrics**. The package structure is a very good second choice. Unlike in an application framework, a library’s packages are designed as the architecture. ACDC and k-means are useful to look *inside* the large packages (e.g. families in `collect`, `util.concurrent`), but not as the top-level architecture. For *evolution* studies ACDC remains the best automatic method: it has the highest course MQ and is the most stable between versions (MoJoFM $\approx 89\%$ vs $\approx 72\%$ for k-means).

5.2 Does the AI tool help? How do information and prompts influence the result?

- **For a well-known, stable system the AI is already good without context (P1).** This is the opposite of what we saw for a young, fast-changing framework (Spring AI), where P1 contained removed modules. The value of extra information is in **precision**, not correctness. With the class list and the dependency graph (P3) the answer becomes a complete, checkable mapping with measured dependencies and cycles.
- **The granularity of the input determines the granularity of the output.** With packages only (P2) the AI cannot see inside `collect`. Only when we gave it the class

names and asked for a class-level mapping did it split the god package, using naming conventions (Immutable*, Regular*, *Multimap).

- **Feeding back metrics (P4) works well.** The AI correctly explained the effect of granularity and proposed a hierarchical view instead of optimising the metric blindly.
- **Risks.** The AI does not measure; every claim (e.g. “only eventbus uses the cache”, “cycles only inside collect”) had to be checked against the extracted data. In its first draft of P4 one estimate (the number of dependencies cut by splitting collect) and one statement about the cache had to be corrected.

5.3 Which method for other systems?

1. **Start from the packages and the AI**, given the extracted facts (class list + dependency graph), for the top-level component view. This is especially effective for libraries and well-known systems.
2. **Use ACDC with noise removal** for *evolution*: it is deterministic and the most stable across versions, and its clusters show the internal families of large packages.
3. **Check the noise set** against fan-in before trusting A2/A4. JNode flagged an entire leaf package (cache) and missed most collection hubs.
4. k-means needs an extra decision (features, k) and is the least stable. It is useful as a second opinion, not as the primary method.

6. Threats to validity

- **Sparse sampling.** Every second major release (gaps of 6 months to 3 years) hides short-term changes and lowers the stability values.
- **Binary analysis** of the -jre jar: reflection and the Android flavour are not covered.
- **Tools.** JNode was patched (performance and precision, identical where the original works) and needed a threshold fallback for 10.0 and 12.0. ACDC is the ARCADE port and needed file-style names (2.3).
- **k-means** depends on the feature representation and on the elbow choice of k. **Cohesion, the course MQ and TurboMQ** depend strongly on k, so TurboMQ/k and the intra share complement the course MQ when k differs strongly.

References

- Mancoridis S. et al. (1998). *Using Automatic Clustering to Produce High-Level System Organizations of Source Code*. IWPC.
- Mitchell B. S., Mancoridis S. (2006). *On the automatic modularization of software systems using the Bunch tool*. IEEE TSE 32(3).
- Tzerpos V., Holt R. C. (2000). *ACDC: An Algorithm for Comprehension-Driven Clustering*. WCRE.
- Lehman M. M. (1996). *Laws of Software Evolution Revisited*. EWSPT.
- Turski W. M. (1996). *Reference model for smooth growth of software systems*. IEEE TSE 22(8).
- Satopää V. et al. (2011). *Finding a “Kneedle” in a Haystack*. ICDCSW.
- Guava README, user guide (wiki) and Javadoc, v33.7.0.

Appendix A - Full metric tables

Table 5 - Number of clusters (k).

Version	A1 ACDC	A2 ACDC-noise	A3 k-means	A4 k-means-noise	Packages
10.0	41	37	52	42	8
12.0	51	51	52	46	11
14.0	70	63	52	49	11
16.0	72	62	60	46	15
18.0	74	62	52	40	15
20.0	80	66	52	48	16
22.0	79	67	74	56	16
24.0	88	76	74	52	16
26.0	89	79	59	64	16
28.0	90	81	54	60	16
30.0	96	87	69	52	16
32.0.0	95	86	59	56	16
33.7.0	90	84	69	64	17

Table 7 - TurboMQ/k.

Version	A1 ACDC	A2 ACDC-noise	A3 k-means	A4 k-means-noise	Packages
10.0	0.399	0.520	0.349	0.526	0.564
12.0	0.341	0.399	0.352	0.461	0.555
14.0	0.305	0.400	0.349	0.436	0.544
16.0	0.312	0.435	0.325	0.512	0.479
18.0	0.321	0.435	0.355	0.544	0.471
20.0	0.312	0.445	0.375	0.525	0.486
22.0	0.309	0.451	0.292	0.517	0.489
24.0	0.298	0.439	0.299	0.531	0.486
26.0	0.302	0.435	0.350	0.501	0.485
28.0	0.296	0.428	0.364	0.519	0.487
30.0	0.287	0.433	0.335	0.539	0.491
32.0.0	0.290	0.389	0.369	0.488	0.491
33.7.0	0.314	0.419	0.331	0.476	0.469

Table 7b - TurboMQ.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	16.4	19.2	18.1	22.1	4.5
12.0	17.4	20.3	18.3	21.2	6.1
14.0	21.4	25.2	18.2	21.4	6.0
16.0	22.5	27.0	19.5	23.5	7.2
18.0	23.7	26.9	18.5	21.8	7.1
20.0	25.0	29.3	19.5	25.2	7.8
22.0	24.4	30.2	21.6	29.0	7.8
24.0	26.2	33.3	22.1	27.6	7.8
26.0	26.8	34.3	20.6	32.0	7.8
28.0	26.7	34.6	19.6	31.2	7.8
30.0	27.6	37.7	23.1	28.0	7.9
32.0.0	27.5	33.4	21.8	27.3	7.9
33.7.0	28.3	35.2	22.9	30.4	8.0

Table 7c – MQ (course definition).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.2507	0.2384	0.1855	0.2061	0.0924
12.0	0.2524	0.2583	0.1759	0.1954	0.1208
14.0	0.2545	0.2506	0.1745	0.1862	0.1121
16.0	0.2599	0.2540	0.1791	0.1812	0.1102
18.0	0.2619	0.2527	0.1711	0.1765	0.1080
20.0	0.2527	0.2490	0.1691	0.1745	0.0993
22.0	0.2508	0.2430	0.1663	0.1833	0.0992
24.0	0.2494	0.2442	0.1608	0.1685	0.0957
26.0	0.2456	0.2414	0.1559	0.1864	0.0956
28.0	0.2486	0.2404	0.1522	0.1897	0.0963
30.0	0.2429	0.2364	0.1702	0.1666	0.0947
32.0.0	0.2479	0.2449	0.1691	0.1727	0.0950
33.7.0	0.2495	0.2423	0.1668	0.1843	0.0842

Table 8a – Cohesion.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.2586	0.2432	0.1937	0.2109	0.0989
12.0	0.2628	0.2675	0.1823	0.1998	0.1259
14.0	0.2613	0.2568	0.1811	0.1929	0.1176
16.0	0.2670	0.2598	0.1871	0.1853	0.1156
18.0	0.2686	0.2580	0.1775	0.1812	0.1133
20.0	0.2594	0.2534	0.1746	0.1779	0.1040
22.0	0.2566	0.2469	0.1741	0.1874	0.1038
24.0	0.2556	0.2480	0.1672	0.1717	0.1005
26.0	0.2511	0.2452	0.1607	0.1903	0.1005
28.0	0.2541	0.2442	0.1572	0.1933	0.1011
30.0	0.2482	0.2399	0.1760	0.1697	0.0994
32.0.0	0.2535	0.2492	0.1748	0.1763	0.0998
33.7.0	0.2542	0.2461	0.1764	0.1879	0.0885

Table 8b – Coupling.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.00787	0.00487	0.00817	0.00480	0.00649
12.0	0.01043	0.00921	0.00640	0.00442	0.00510
14.0	0.00684	0.00614	0.00655	0.00667	0.00551
16.0	0.00706	0.00576	0.00802	0.00413	0.00539
18.0	0.00664	0.00529	0.00637	0.00461	0.00533
20.0	0.00666	0.00433	0.00540	0.00335	0.00475
22.0	0.00588	0.00392	0.00775	0.00409	0.00468
24.0	0.00628	0.00386	0.00640	0.00323	0.00481
26.0	0.00549	0.00379	0.00483	0.00386	0.00481
28.0	0.00553	0.00385	0.00498	0.00363	0.00482
30.0	0.00540	0.00353	0.00581	0.00303	0.00473
32.0.0	0.00560	0.00426	0.00573	0.00360	0.00473
33.7.0	0.00475	0.00375	0.00959	0.00356	0.00425

Table 8c – Share of dependencies inside clusters.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
10.0	0.427	0.555	0.302	0.513	0.773
12.0	0.398	0.432	0.331	0.398	0.738
14.0	0.359	0.412	0.295	0.346	0.727
16.0	0.360	0.462	0.266	0.449	0.718
18.0	0.367	0.463	0.297	0.468	0.713
20.0	0.366	0.505	0.335	0.464	0.697
22.0	0.366	0.492	0.250	0.431	0.695
24.0	0.352	0.480	0.262	0.448	0.696
26.0	0.357	0.473	0.310	0.425	0.699
28.0	0.358	0.469	0.319	0.443	0.702
30.0	0.352	0.482	0.286	0.505	0.701
32.0.0	0.359	0.431	0.306	0.418	0.705
33.7.0	0.359	0.461	0.286	0.419	0.720

Table 8d – Stability (ARI with the previous sampled version).

Version (vs previous)	A1	A2	A3	A4
12.0	0.7594	0.7368	0.616	0.4722
14.0	0.4692	0.5891	0.5186	0.5066
16.0	0.7095	0.6562	0.4807	0.4941
18.0	0.895	0.8258	0.5381	0.5583
20.0	0.7197	0.8387	0.5037	0.5363
22.0	0.7625	0.7722	0.4959	0.6564
24.0	0.8291	0.7999	0.6166	0.6718
26.0	0.9645	0.9617	0.6119	0.5981
28.0	0.9162	0.8921	0.6858	0.7335
30.0	0.8981	0.8068	0.6348	0.6881
32.0.0	0.9421	0.7867	0.6122	0.6066
33.7.0	0.8128	0.747	0.5698	0.5603

Table 9 – Package and class changes between sampled versions.

Version	Packages	Pkgs added	Pkgs removed	Classes added	Classes removed
12.0	13	3	0	63	9
14.0	13	0	0	58	23
16.0	17	4	0	50	14
18.0	17	0	0	8	4
20.0	18	1	0	76	16
22.0	18	0	0	20	5
24.0	18	0	0	24	4
26.0	18	0	0	7	0
28.0	18	0	0	2	4
30.0	18	0	0	19	5
32.0.0	18	0	0	6	1
33.7.0	18	0	0	14	18

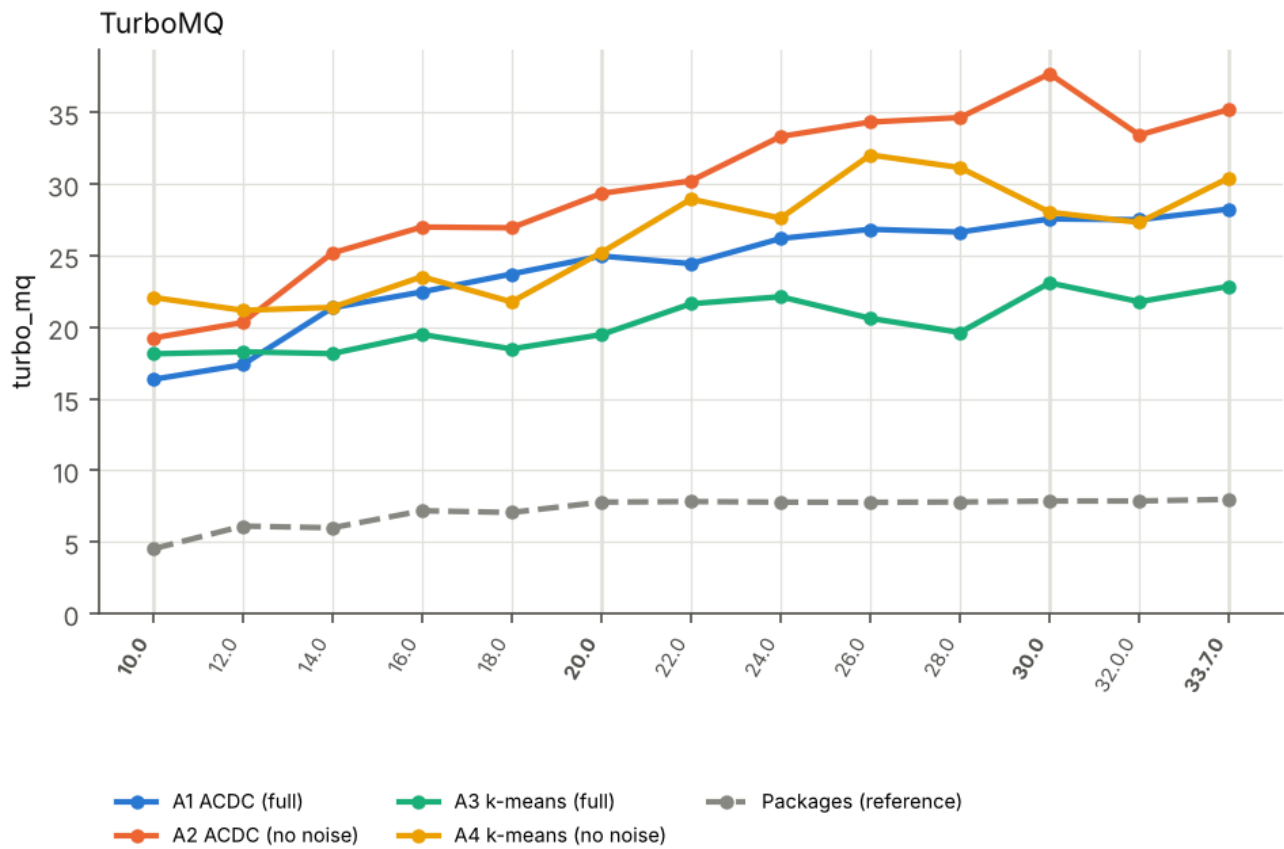


Figure A1 - TurboMQ.

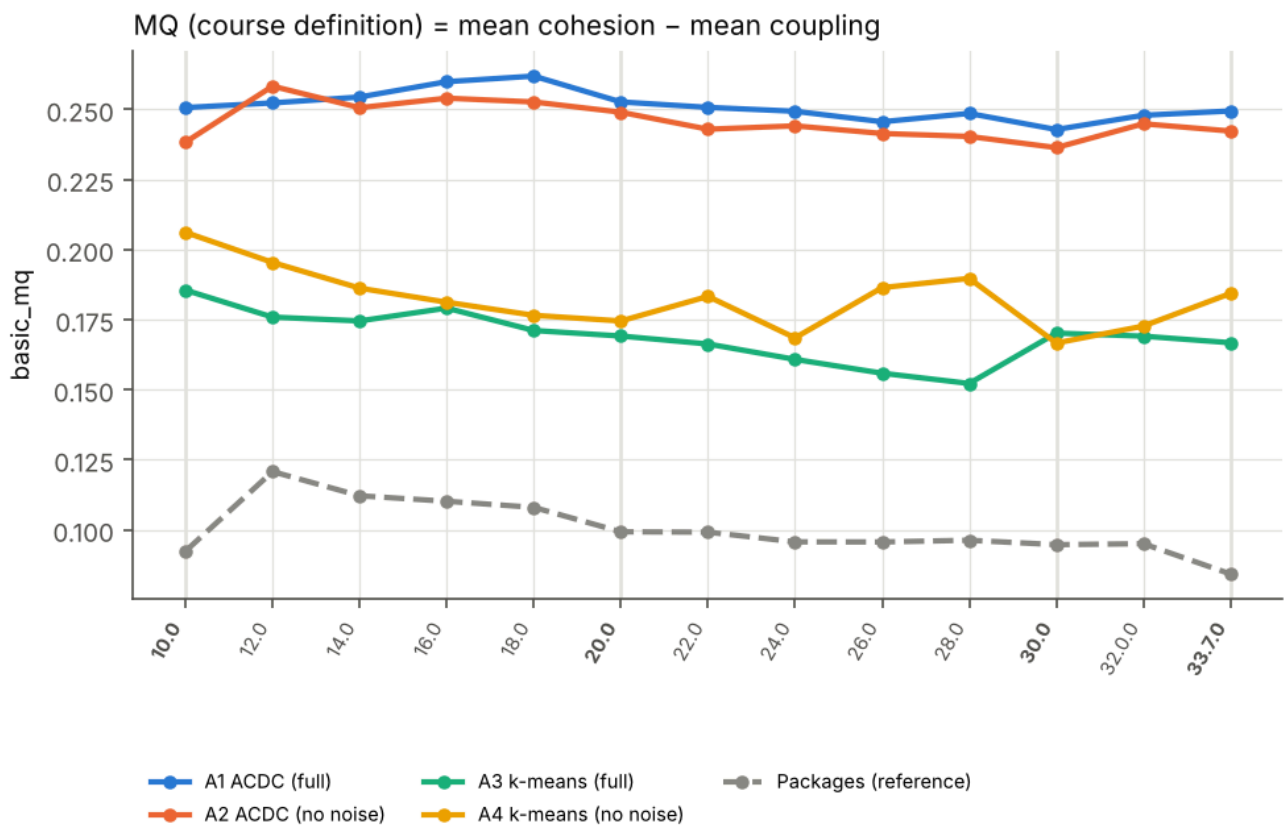


Figure A2 - MQ (course definition).